

UNIVERSIDAD DE SALAMANCA

FACULTAD DE CIENCIAS

DEPARTAMENTO DE ESTADÍSTICA



VNiVERSiDAD  
D SALAMANCA

CAMPUS DE EXCELENCIA INTERNACIONAL

TRABAJO DE FIN DE GRADO

OPERADORES NEURONALES:  
FUNDAMENTOS TEÓRICOS Y  
APLICACIÓN A LA  
RESOLUCIÓN DE ECUACIONES  
DIFERENCIALES

Tutor: Miguel Rodríguez Rosa

Autor: Alberto Martín Marcos

Grado en Matemáticas

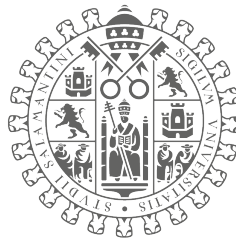
Curso académico 2025-2026



UNIVERSIDAD DE SALAMANCA

FACULTAD DE CIENCIAS

DEPARTAMENTO DE ESTADÍSTICA



VNiVERSiDAD  
D SALAMANCA

CAMPUS DE EXCELENCIA INTERNACIONAL

TRABAJO DE FIN DE GRADO

OPERADORES NEURONALES:  
FUNDAMENTOS TEÓRICOS Y  
APLICACIÓN A LA  
RESOLUCIÓN DE ECUACIONES  
DIFERENCIALES

Firmado por:

Autor: Alberto Martín Marcos

Tutor: Miguel Rodríguez Rosa

Grado en Matemáticas

Curso académico 2025-2026



*A mi madre, mi hermana y Lorena, mis tres grandes apoyos.*

# Resumen

Los Operadores Neuronales (*Neural Operators*) constituyen un marco matemático orientado al aprendizaje de transformaciones entre espacios de funciones, con el objetivo de aproximar operadores que actúan sobre soluciones de ecuaciones diferenciales. A diferencia de los modelos neuronales tradicionales, que aprenden funciones de dimensión finita, los operadores neuronales se formulan como aproximadores universales de operadores no lineales entre espacios de funciones, lo que permite modelar directamente la dependencia entre un conjunto de datos iniciales, o coeficientes, y la solución completa de un problema de ecuaciones diferenciales. Entre las arquitecturas más relevantes se encuentran los Operadores Neuronales de Fourier (*Fourier Neural Operators*, FNO), que realizan las actualizaciones principales del operador en el dominio de las frecuencias mediante transformadas discretas de Fourier, proporcionando una aproximación eficiente y capaz de generalizar en mallas espaciales distintas a las empleadas durante el entrenamiento.

El trabajo propuesto consistirá, en primer lugar, en una exposición rigurosa de los fundamentos matemáticos de los operadores neuronales, incluyendo su formulación funcional, sus propiedades de aproximación y su relación con los métodos numéricos clásicos para ecuaciones diferenciales. Se desarrollará posteriormente un capítulo dedicado específicamente a los Operadores Neuronales de Fourier, detallando su estructura algorítmica, su interpretación y los resultados teóricos que justifican su aplicabilidad.

A continuación, se abordará el estudio de varios problemas clásicos de la física matemática formulados como ecuaciones diferenciales, tales como la ecuación de Navier–Stokes, la ecuación de Burgers y otros sistemas lineales o no lineales. Para cada uno de ellos se presentará la formulación correspondiente, así como su planteamiento como problema de operador que puede ser aproximado mediante operadores neuronales. El objetivo será analizar de forma crítica cómo pueden emplearse los operadores neuronales, y en particular los operadores de Fourier, para aproximar las soluciones de estos modelos, comparando conceptualmente este enfoque con los métodos numéricos tradicionales.

Finalmente, el trabajo incluirá un capítulo dedicado a la implementación computacional en Python de los métodos estudiados. Se desarrollará un código que permita entrenar y evaluar operadores neuronales para la resolución de algunos de los problemas seleccionados, documentando de forma detallada la arquitectura empleada, la preparación de los datos, los criterios de entrenamiento y las métricas utilizadas para validar los resultados, lo que permitirá ilustrar empíricamente el rendimiento del método y apoyar las conclusiones derivadas del estudio teórico.

## Summary

Neural Operators constitute a mathematical framework oriented toward learning mappings between function spaces, with the objective of approximating operators that act upon solutions to differential equations. Unlike traditional neural models, which learn finite-dimensional functions, neural operators are formulated as universal approximators of nonlinear operators between function spaces. This allows for the direct modeling of the dependence between a set of initial data (or coefficients) and the full solution of a differential equation problem. Among the most prominent architectures are Fourier Neural Operators (FNO), which perform the operator’s primary updates in the frequency domain via discrete Fourier transforms. This approach provides an efficient approximation capable of generalizing across spatial meshes different from those utilized during training.

The proposed work will consist, first, of a rigorous exposition of the mathematical foundations of neural operators, including their functional formulation, approximation properties, and their relationship with classical numerical methods for differential equations. Subsequently, a chapter will be dedicated specifically to Fourier Neural Operators, detailing their algorithmic structure, interpretation, and the theoretical results justifying their applicability.

Next, the study will address several classical problems in mathematical physics formulated as differential equations, such as the Navier–Stokes equations, Burger’s equation, and other linear or nonlinear systems. For each, the corresponding formulation will be presented, alongside its framing as an operator problem amenable to approximation via neural operators. The objective is to critically analyze how neural operators—and Fourier operators in particular—can be employed to approximate the solutions of these models, conceptually comparing this approach with traditional numerical methods.

Finally, the work will include a chapter dedicated to the computational implementation of the studied methods in Python. Code will be developed to train and evaluate neural operators for solving selected problems, providing detailed documentation of the architecture employed, data preprocessing, training criteria, and the metrics used to validate the results. This will allow for an empirical illustration of the method’s performance and support the conclusions derived from the theoretical study.

# Índice general

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| <b>Introducción</b>                                                   | <b>1</b>  |
| <b>1. Fundamentos Matemáticos de los Operadores Neuronales</b>        | <b>3</b>  |
| El Problema Matemático . . . . .                                      | 3         |
| El Problema Computacional . . . . .                                   | 5         |
| Recordatorio sobre Redes Neuronales Clásicas . . . . .                | 6         |
| Operadores Neuronales: aprendizaje en espacios de funciones . . . . . | 14        |
| Ventajas de los Operadores Neuronales . . . . .                       | 18        |
| Aproximación Universal de Operadores Neuronales . . . . .             | 20        |
| Invarianza respecto a la Discretización . . . . .                     | 25        |
| Comparativa con otros Métodos Numéricos Clásicos . . . . .            | 29        |
| <b>2. Operadores Neuronales de Fourier</b>                            | <b>31</b> |
| Convolución en el dominio de las frecuencias . . . . .                | 32        |
| El caso del dominio discreto . . . . .                                | 34        |
| Invarianza a la resolución . . . . .                                  | 35        |
| Extensión a otras geometrías del dominio . . . . .                    | 37        |
| <b>3. Problemas Clásicos en Física Matemática</b>                     | <b>38</b> |
| Ecuación de Poisson . . . . .                                         | 38        |
| Ecuación de Burgers . . . . .                                         | 40        |
| Ecuación de Navier–Stokes . . . . .                                   | 41        |
| <b>4. Programación de un Operador Neuronal</b>                        | <b>44</b> |
| Carga y preparación de la base de datos . . . . .                     | 44        |
| Entrenamiento del modelo . . . . .                                    | 47        |
| Evaluación de las Predicciones . . . . .                              | 51        |
| <b>5. Conclusiones</b>                                                | <b>54</b> |
| <b>Bibliografía</b>                                                   | <b>54</b> |
| <b>Anexo</b>                                                          | <b>57</b> |

# Introducción

## Estado de la cuestión

Las ecuaciones en derivadas parciales (EDP) se utilizan para modelar fenómenos y sistemas en ramas de la ciencia muy diversas, como la biología, la física o las finanzas estocásticas. Encontrar soluciones analíticas para este tipo de ecuaciones suele ser muy complicado, es por esto que a lo largo de la historia de las Matemáticas se han desarrollado métodos de carácter numérico (como el Método de los Elementos Finitos o el de las Diferencias Finitas) tratando de aproximar su solución, siempre con un margen de error. Sin embargo, los cálculos científicos suelen ser muy costosos en términos de tiempo y recursos. Por ejemplo, ejecutar simulaciones numéricas de dinámica de fluidos o sistemas de muchos cuerpos puede tardar días o incluso meses. Este alto coste se debe al aumento del número de los cálculos necesarios a medida que discretizamos en cuadrículas más pequeñas para lograr una precisión aceptable.

En este contexto, la cada vez más amplia aplicación en los últimos años de modelos de Aprendizaje Automático, como las Redes Neuronales, ha llamado la atención de los investigadores debido a su poderosa capacidad de aproximación universal, surgiendo como alternativas a los solucionadores numéricos tradicionales. Aunque estos nuevos métodos basados en datos han demostrado su utilidad, presentan problemas inherentes a su estructura: las Redes Neuronales son aplicaciones entre espacios de dimensión finita. Esta característica hace que los modelos solo puedan ser entrenados en una sola discretización, lo que puede presentar dificultades, pues la predicción de la Red puede quedar sesgada por la malla concreta utilizada durante el entrenamiento.

Para resolver este inconveniente, surgió la idea de reformular estos problemas como problemas de aproximación de operadores, donde el objetivo ya no es aprender una aplicación entre espacios vectoriales de dimensión finita, sino una transformación entre espacios de funciones (espacios de dimensión infinita). Este enfoque resulta más realista, ya que los datos de una EDP no son originalmente vectores, sino funciones definidas sobre un dominio.

El origen de esta idea se remonta a los trabajos de Chen y Chen [5] en los años noventa, donde se estudió la aproximación universal de operadores no lineales mediante redes neuronales. Sin embargo, su desarrollo no llegó hasta décadas después. En 2019, Lu, Jin y Karniadakis propusieron DeepONet [20], una de las primeras arquitecturas diseñadas específicamente para aprender operadores entre espacios de funciones.

Sobre esta base, en 2021, comenzó a construirse la teoría de **Operadores Neuronales**. Los Operadores Neuronales son arquitecturas basadas en Redes Neuronales cuyas entradas y salidas son funciones en lugar de vectores. Bajo ciertas hipótesis, los Operadores Neuronales también poseen capacidad de aproximación universal de operadores

continuos. Esta propiedad permite interpretarlos como una extensión de las redes neuronales clásicas al caso infinito-dimensional, permitiendo aproximar operadores solución de ecuaciones diferenciales en lugar de aplicaciones entre vectores discretizados.

## Objetivos

En este trabajo tendremos los siguientes objetivos:

- Desarrollar la teoría de los Operadores Neuronales: formulación funcional, teoremas de aproximación y su relación con métodos numéricos clásicos para la resolución de EDP.
- Estudiar en particular las propiedades del Operador Neuronal de Fourier (FNO).
- Plantear algunos problemas clásicos de la física usando este nuevo paradigma.
- Programar una red neuronal para la resolución de dichos problemas.

## Estructura

Una vez definidos los términos principales de este trabajo, veremos su estructura:

Nuestro primer capítulo, presenta una introducción a la teoría de Redes Neuronales, que nos será útil para desarrollar conceptos posteriores. Seguidamente, veremos cómo podemos resolver el problema de aproximación de operadores entre espacios de funciones en mediante técnicas de Aprendizaje Automático. Después, estudiaremos sus principales características: la invarianza a la discretización y la aproximación universal. Terminaremos con una comparativa de estos modelos con otros basados en Redes Neuronales y Métodos Numéricos Clásicos.

Nuestro segundo capítulo hablará de una arquitectura de modelo perteneciente a la familia de los Operadores Neuronales: el Operador Neuronal de Fourier. Estudiaremos su formulación y sus ventajas frente a otras arquitecturas.

En nuestro tercer capítulo, reformularemos ciertos problemas famosos de ecuaciones en derivadas parciales como problemas de aproximación de su operador solución, que puede ser aproximado mediante operadores neuronales.

El cuarto capítulo estará dedicado a la implementación computacional de esta nueva arquitectura, entrenaremos un modelo basado en operadores neuronales y comprobaremos con diversas métricas su fiabilidad y capacidad de inferencia.

Finalmente, en nuestro capítulo final se encuentran detalladas las conclusiones obtenidas a partir del desarrollo y análisis realizados. Por último, se ofrecerá la bibliografía utilizada para la realización de este trabajo, en esta encontraremos diversos libros, artículos, y páginas web.

# Capítulo 1

## Fundamentos Matemáticos de los Operadores Neuronales

### El Problema Matemático

Sea  $D \subset \mathbb{R}^n$  un subconjunto abierto y acotado y sean  $\mathcal{A} = L^\infty(D; \mathbb{R}_+)$  y  $\mathcal{U} = H_0^1(D; \mathbb{R})$ , ambos espacios de Banach. Consideremos la siguiente familia de ecuaciones diferenciales en derivadas parciales:

$$\begin{cases} (L_a u)(x) = f(x), & x \in D, \\ u(x) = 0, & x \in \partial D. \end{cases}$$

donde  $a \in \mathcal{A}$  es un parámetro funcional,  $\mathcal{U} \ni u : D \rightarrow \mathbb{R}$  la solución de la ecuación diferencial<sup>1</sup> y  $f \in \mathcal{U}^* = H^{-1}(D; \mathbb{R})$ <sup>2</sup>.

Siendo  $L$  una aplicación entre el espacio de parámetros  $\mathcal{A}$  y el espacio de operadores lineales (posiblemente no acotados) entre  $\mathcal{U}$  y su dual:

$$\begin{aligned} L : \mathcal{A} &\longrightarrow \mathcal{L}(\mathcal{U}, \mathcal{U}^*) \\ a &\longmapsto L_a \end{aligned}$$

Dar una solución de la ecuación implica dar la función  $u \in \mathcal{U}$  de modo que  $L_a(u) = f$ , así pues,  $(L_a)^{-1}(f) = u$ , o equivalentemente  $(L^{-1}f)(a) = u$ . Un operador natural derivado de este razonamiento es  $\mathcal{G}^\dagger := L^{-1}f : \mathcal{A} \rightarrow \mathcal{U}$ , que asocia cada parámetro a su solución  $a \mapsto u$  [16].

En este caso, podemos definir la función de Green  $G : D \times D \rightarrow \mathbb{R}$  asociada a este problema como la única solución de la ecuación

$$L_a G(x, \cdot) = \delta_x$$

---

<sup>1</sup>Notemos que  $u \in \mathcal{U}$  es única fijando la condición  $u(x) = 0$  en  $\partial D$ , el Teorema de Existencia y Unidad lo garantiza.

<sup>2</sup>La función  $f : D \rightarrow \mathbb{R}$  se interpreta como un elemento del dual del espacio de soluciones  $\mathcal{U}$ . Así pues,  $f$  puede verse como el operador lineal

$$\begin{aligned} f : \mathcal{U} &\longrightarrow \mathbb{R} \\ v &\longmapsto \int_D f(x) v(x) dx \end{aligned}$$

donde  $\delta_x$  es la delta de Dirac o medida concentrada en el punto  $x \in D$ . Notemos que  $G$  dependerá del parámetro  $a$ , por lo que en lo sucesivo la denotaremos como  $G_a$ . La solución de la ecuación diferencial puede entonces representarse como

$$u(x) = \int_D G_a(x, y) f(y) dy.$$

Esto se ve a través del cálculo [18]

$$(L_a u)(x) = \int_D (L_a G(x, \cdot))(y) f(y) dy = \int_D \delta_x(y) f(y) dy = f(x)$$

Rigurosamente,  $\delta_x$  se define como la distribución [10] (o funcional lineal perteneciente al espacio dual  $\mathcal{U}^*$ ) que actúa sobre cualquier función de prueba  $v \in U$  mediante la propiedad de evaluación:

$$\langle \delta_x, v \rangle = \int_D \delta(y - x) v(y) dy = v(x)$$

que puede representarse heurísticamente como

$$\delta_x(y) \approx \begin{cases} \infty, & y - x = 0, \\ 0, & y - x \neq 0. \end{cases}$$

tal que  $\int_{-\infty}^{\infty} \delta(x) dx = 1$ . Dado que ninguna función posee esta propiedad, modelar rigurosamente la *función* delta implica el uso de límites o, como es común en matemáticas, la teoría de la medida y la teoría de las distribuciones.

De este modo,  $\delta_x$  modela una fuente puntual idealizada en  $x$ , permitiendo reconstruir cualquier término fuente  $f \in U^*$  mediante el principio de superposición. En el caso particular de la delta de Dirac  $\delta_x$ , la distribución actúa evaluando la función en el punto  $x$ , es decir,  $T_{\delta_x}(v) = v(x)$ . Al aplicar la identidad anterior, se identifica que la medida asociada <sup>3</sup> es la medida de Dirac  $\delta_x(A)$ , la cual concentra toda la masa probabilística en el punto  $x$ :

$$v(x) = \int_D v(y) d\delta_x(y)$$

A pesar de todo esto, no siempre vamos a poder conocer el operador  $\mathcal{G}^\dagger$ . Por este motivo, nuestra empresa consistirá en aproximarlos.

Este resultado es muy revelador y es el núcleo de este trabajo. Hemos transformado un problema de aproximación de operadores en uno de aproximación de funciones (la función de Green asociada al operador la EDP). Esto nos permitirá aplicar las técnicas de la teoría de redes neuronales, puesto que las redes neuronales son excelentes aproximadores de funciones.

---

<sup>3</sup>Para obtener la medida asociada a una distribución, se establece una correspondencia biunívoca a través de la acción del funcional sobre el espacio de funciones de prueba continuas con soporte compacto,  $C_c(D)$ . Específicamente, si una distribución  $T$  es positiva, existe una única medida de Borel  $\mu$  en  $D$  tal que para toda  $v \in C_c(D)$ :

$$T(v) = \int_D v(y) d\mu(y)$$

## El Problema Computacional

Nuestro objetivo es aproximar una aplicación entre espacios de funciones (dimension infinita) usando una conjunto finito de observaciones de pares de entrada-salida de  $\mathcal{G}^\dagger$ . Sean  $\mathcal{A}$  y  $\mathcal{U}$  espacios de Banach de funciones definidas sobre dominios acotados  $D \subset \mathbb{R}^n$  y  $D' \subset \mathbb{R}^{n'}$  respectivamente y  $\mathcal{G}^\dagger : \mathcal{A} \rightarrow \mathcal{U}$  un operador generalmente no lineal.

Suponemos dadas observaciones  $\{a^{(i)}, u^{(i)}\}_{i=1}^N$ , donde  $a^{(i)} \in \mathcal{A}$  son muestras independientes tomadas de una distribución de probabilidad  $\mu$  definida sobre  $\mathcal{A}$  y  $u^{(i)} = \mathcal{G}^\dagger(a^{(i)})$ , que puede estar perturbado por el ruido.

Nos proponemos construir una aproximación de  $\mathcal{G}^\dagger$  a través de una familia paramétrica de operadores:

$$\mathcal{G}_\theta : \mathcal{A} \longrightarrow \mathcal{U}, \quad \theta \in \mathbb{R}^p$$

siendo los parámetros elementos de un  $\mathbb{R}$ -espacio vectorial de dimensión finita y escogiendo el  $\theta^\dagger \in \mathbb{R}^p$  tal que  $\mathcal{G}_{\theta^\dagger} \approx \mathcal{G}^\dagger$ .

Nos interesará analizar el error de aproximación promedio con respecto a  $\mu$ . En particular, suponiendo que  $\mathcal{G}^\dagger$  es  $\mu$ -medible, evaluaremos el error de aproximación mediante la norma de Bochner  $L_\mu^2(\mathcal{A}; \mathcal{U})$  (norma  $L^2$  con respecto a la medida  $\mu$ ).

$$\|\mathcal{G}^\dagger - \mathcal{G}_\theta\|_{L_\mu^2(\mathcal{A}; \mathcal{U})}^2 = \mathbb{E}_{a \sim \mu} \|\mathcal{G}^\dagger(a) - \mathcal{G}_\theta(a)\|_{\mathcal{U}}^2 = \int_{\mathcal{A}} \|\mathcal{G}^\dagger(a) - \mathcal{G}_\theta(a)\|_{\mathcal{U}}^2 d\mu(a).$$

Siendo  $X = X_i := \|\mathcal{G}^\dagger(a) - \mathcal{G}_\theta(a)\|_{\mathcal{U}}^2$  variables aleatorias *i.i.d* sobre el espacio de probabilidad  $(\mathcal{A}, \mu)$ , por la Ley de los Grandes Números, sea  $\bar{X}_n = \frac{X_1 + \dots + X_n}{n}$ , la sucesión  $\bar{X}_n$  converge en probabilidad a  $\mathbb{E}_{a \sim \mu}[X_i]$ , es decir, para todo  $\varepsilon > 0$ , se cumple que

$$\lim_{n \rightarrow \infty} \mathbb{P}\left(\left|\bar{X}_n - \mathbb{E}_{a \sim \mu}[X_i]\right| < \varepsilon\right) = 1.$$

De esta forma, para un  $N$  suficientemente grande

$$\frac{1}{N} \sum_{i=1}^N \|u^{(i)} - \mathcal{G}_\theta(a^{(i)})\|_{\mathcal{U}}^2 = \frac{1}{N} \sum_{i=1}^N X_i(a^{(i)}) \approx \mathbb{E}_{a \sim \mu} \|\mathcal{G}^\dagger(a) - \mathcal{G}_\theta(a)\|_{\mathcal{U}}^2$$

Esto constituye un marco natural para el aprendizaje en espacios de dimensión infinita, ya que se puede plantear el problema asociado de minimización del error empírico [22]

$$\min_{\theta \in \mathbb{R}^p} \mathbb{E}_{a \sim \mu} \|\mathcal{G}^\dagger(a) - \mathcal{G}_\theta(a)\|_{\mathcal{U}}^2 \approx \min_{\theta \in \mathbb{R}^p} \frac{1}{N} \sum_{i=1}^N \|u^{(i)} - \mathcal{G}_\theta(a^{(i)})\|_{\mathcal{U}}^2$$

lo cual es directamente análogo al caso clásico de dimensión finita.

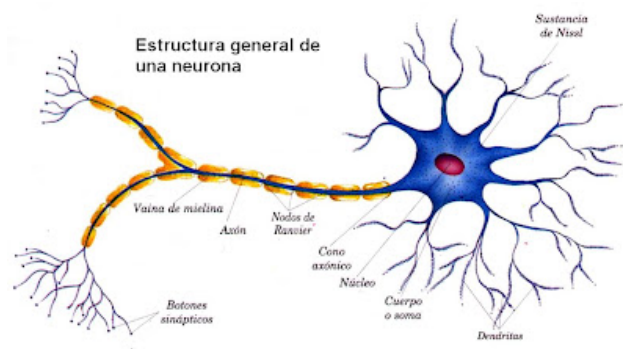
Además de utilizar el error medido mediante la norma de Bochner, también consideraremos el caso en el que el error se mide uniformemente sobre subconjuntos compactos de  $\mathcal{A}$ . En particular, dado cualquier  $K \subset \mathcal{A}$  compacto,

$$\sup_{a \in K} \|\mathcal{G}^\dagger(a) - \mathcal{G}_\theta(a)\|_{\mathcal{U}}$$

## Recordatorio sobre Redes Neuronales Clásicas

El aprendizaje automático (*Machine Learning, ML*) se refiere al campo de estudio de los algoritmos que infieren, a partir de datos dados, cómo realizar una tarea específica sin haber sido programados explícitamente para ello (Arthur Samuel, 1959). Estos algoritmos han logrado éxitos increíbles dentro de ámbitos como el reconocimiento de imágenes, el procesamiento de lenguaje natural [9], y han realizado hazañas nunca antes vistas como derrotar al campeón del mundo de Ajedrez, como hizo el software Deep Blue [2]. Aquí consideramos un subconjunto popular de algoritmos de ML: **las Redes Neuronales**.

La inspiración de las Redes Neuronales (NN) proviene de una analogía con el cerebro humano, donde las células biológicas (neuronas) responden a la activación de otras neuronas a las que están conectadas. A un nivel muy básico, las neuronas del cerebro reciben señales eléctricas que luego se transmiten a sus salidas. La activación de esta relación también depende de la intensidad de la conexión, es decir, una neurona puede ser más o menos sensible a otra, y así sucesivamente.

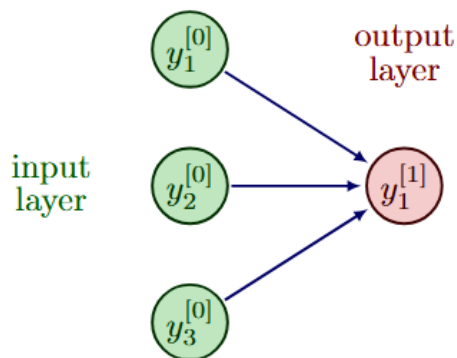


**Figura 1.1:** Estructura de una neurona cerebral [11]

Para una única neurona/nodo con entrada  $u \in \mathbb{R}^n$ , un modelo matemático, denominado *perceptrón*, puede describirse como

$$y = \sigma \left( \sum_{i=1}^n W_i u_i + b \right) = \sigma \left( W^\top u + b \right),$$

donde  $y$  es la **activación** de la neurona/nodo,  $W_i$  son los pesos y  $b$  es el sesgo [9].



**Figura 1.2:** Representación esquemática del perceptrón con una entrada tridimensional. Denotamos la entrada por  $y^{[0]} = u$  y la salida por  $y^{[1]} = y$ . Los pesos  $W_i$  se aplican sobre las flechas y el sesgo se añade en el nodo  $y_1^{[1]}$ .

La función  $\sigma : \mathbb{R} \rightarrow \mathbb{R}$  se denomina *función de activación*. El argumento de la función de activación siempre es la transformación afín  $z = \mathbf{w} \cdot \mathbf{x} + b$ , al que se suele llamar *suma ponderada*. Originalmente, se propuso elegir la función de Heaviside como función

de activación para modelar si una neurona se activa o no, es decir,

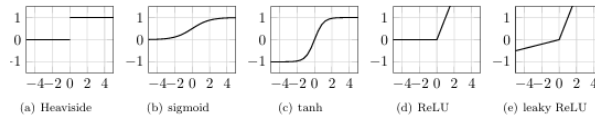
$$\sigma(y) = \begin{cases} 1 & \text{si } y \geq 0, \\ 0 & \text{si } y < 0. \end{cases}$$

Sin embargo, con el tiempo se han propuesto y utilizado muchas otras funciones de activación. Normalmente, son funciones monótonas crecientes para mantener el espíritu de la idea original, pero continuas.

Algunas funciones de activación habituales son

|                      |                                                                        |
|----------------------|------------------------------------------------------------------------|
| Sigmoide (logística) | $\sigma(y) = \frac{1}{1 + \exp(-y)}$                                   |
| Tangente hiperbólica | $\sigma(y) = \tanh(y) = \frac{\exp(y) - \exp(-y)}{\exp(y) + \exp(-y)}$ |
| ReLU                 | $\sigma(y) = \max\{y, 0\}$                                             |
| Leaky ReLU           | $\sigma(y) = \max\{\alpha y, y\}$                                      |

**Tabla 1.1:** Funciones de activación más usadas



**Figura 1.3:** Representación de las activaciones más populares

**Observación 1.** La no linealidad de las funciones de activación es una parte esencial del éxito de las redes neuronales. Dado que la concatenación de funciones lineales vuelve a dar lugar a una función lineal, la complejidad que puede alcanzarse utilizando funciones de activación lineales es limitada.

Aunque la función sigmoide aproxima de forma continua a la función de Heaviside y es diferenciable, contiene una operación exponencial que resulta computacionalmente costosa. Problemas similares aparecen con la función tangente hiperbólica. No obstante, el hecho de que  $\tanh$  esté más próxima a la función identidad suele ayudar a acelerar la convergencia, ya que se comporta como un modelo lineal cuando los valores están cerca de cero. Otro reto que debe superarse es el de las derivadas que tienden a anularse, fenómeno claramente presente en las funciones de Heaviside, sigmoide y tangente hiperbólica. En contraste, ReLU no está acotada en valores positivos y, además, resulta relativamente barata de computar, ya que las operaciones lineales suelen estar muy optimizadas en la computación moderna.

## Definición de una Red Neuronal

Matemáticamente, podemos definir a una red neuronal como una función

$$f(\mathbf{x}, \theta),$$

o tal que

$$f : \mathbb{R}^{n_0} \times \mathbb{R}^{n_\theta} \rightarrow \mathbb{R}^{n_L}.$$

La red neuronal dependerá de unos ciertos parámetros internos  $\theta$  (los pesos y sesgos de cada neurona) y de unos datos de entrenamiento  $\mathbf{y}(\mathbf{x})$ , nuestro propósito es obtener su salida para una nueva entrada  $\mathbf{x}$  [9].

Cada neurona tiene como parámetros asociados  $n$  pesos, siendo  $n$  el número de entradas, un sesgo y la función  $\sigma(\cdot)$  que calcula la salida de la neurona, siendo su valor la activación de la neurona.

Con la ayuda de redes neuronales se pretende resolver una tarea. El rendimiento del algoritmo para la tarea dada se mide mediante la función de pérdida  $\mathcal{L}$ , que debe modelarse adecuadamente. Siendo  $f$  la red neuronal, las variables que se aprenden son los pesos  $W$  y los sesgos  $b$  de la red. Por tanto, podemos formular el siguiente problema de optimización [13]:

$$\min_{W,b} \mathcal{L}(y, u, W, b) \quad \text{tal que} \quad y = f(u, W, b).$$

Una posible elección de  $f$  ya se ha dado con el perceptrón, aunque existen otras arquitecturas. Comentaremos algunas de ellas más adelante. Todas ellas tienen en común que contienen pesos y sesgos, por lo que la formulación anterior del problema sigue siendo válida.

Las tareas de aprendizaje pueden dividirse en dos subgrupos: aprendizaje supervisado y no supervisado<sup>4</sup>. Antes de pasar a distintas arquitecturas de redes, abordaremos el modelado de la función de pérdida.

## Algoritmos de optimización

Para el caso del aprendizaje supervisado, definimos  $\theta$ , que agrupa todas las variables, es decir, los pesos  $W$  y el sesgo  $b$ , y escribimos la función de pérdida como

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}^{(i)}(\theta),$$

la cual queremos minimizar, por ejemplo, el error cuadrático medio (MSE) entre la salida predicha por el modelo y nuestro dataset de entradas y salidas reales de tamaño  $N$ . Aquí,  $\mathcal{L}^{(i)}$  denota la función de pérdida evaluada en el dato  $i$ :

$$\mathcal{L}^{(i)}(\theta) = \frac{1}{2} \|y^{(i)} - S(u^{(i)})\|^2.$$

siendo  $u$  el dato de entrada y  $S(u)$  su etiqueta o salida real, que queremos aprender minimizando todo lo posible su diferencia con la salida  $y$ . Recordemos ahora el algoritmo de descenso de gradiente, también conocido como descenso más pronunciado o descenso por lotes.

Posibles criterios de parada incluyen, por ejemplo, fijar un número máximo de iteraciones  $k$ , alcanzar una cierta precisión  $\|\mathcal{L}(\theta)\| < \varepsilon$  con un  $\varepsilon > 0$  pequeño, o bien que se cumpla  $\|\theta^{k+1} - \theta^k\| < \varepsilon$ . A cada iteración del algoritmo se la llama época (*epoch*).

---

<sup>4</sup>Los métodos de aprendizaje llamado 'no supervisado' no usan datos etiquetados, sino que tratan los objetos de entrada como un conjunto de variables aleatorias para modelar su función de densidad de probabilidad, agrupándolos o simplificándolos según cómo se distribuyen y se parecen entre sí en el espacio.

**Algorithm 1** Descenso de Gradiente

---

**Require:** Punto inicial  $\theta^0$ , Tamaño de paso  $\tau > 0$ , Contador  $k = 0$   
**while** no se cumpla el criterio de parada **do**  
     $\theta^{k+1} \leftarrow \theta^k - \tau \cdot \nabla \mathcal{L}(\theta^k)$   
     $k \leftarrow k + 1$   
**end while**

---

La elección de un tamaño de paso adecuado es fundamental para el éxito del método de descenso de gradiente, especialmente porque este tamaño  $\tau$  es el mismo para todas las componentes de  $\theta$ . Puede ocurrir que en algunas componentes la dirección de descenso solo proporcione una mejora en una pequeña vecindad, requiriendo por tanto un tamaño de paso pequeño  $\tau$ . En estos casos, sería posible emplear un algoritmo de búsqueda lineal, como el Método de Newton [4], aunque esto no es habitual en aprendizaje automático.

En la práctica, se suele elegir un tamaño de paso pequeño, esperando que no sea demasiado grande para ninguna componente de  $\theta$ , y posteriormente se puede ajustar de forma adaptativa. En el contexto de aprendizaje automático, al tamaño de paso se le denomina tasa de aprendizaje (*learning rate*).

## Redes Neuronales Prealimentadas

La introducción de capas ocultas, es decir, capas entre la capa de entrada y la capa de salida, conduce a las Redes Neuronales Prealimentadas, también llamadas perceptrones multicapa (MLPs).

Básicamente, el perceptrón de una sola capa se compone con otras capas,  $\ell = 0, \dots, L$ , donde cada perceptrón toma la salida del perceptrón anterior como entrada. Una FNN se define como aquella que tiene capas ocultas. El número de capas  $L$  se denomina *profundidad* de la red, mientras que el número de neuronas por capa es la *anchura* de la red.

Se denominan redes neuronales prealimentadas (*feedforward neural networks*, FNN), ya que la información siempre se mueve en la misma dirección, y de una capa a la siguiente. También se suele referir a las capas de este tipo de redes como *completamente conectadas*, porque todas las neuronas de capas consecutivas están conectadas entre sí. En otras arquitecturas, como en las redes neuronales recurrentes o redes neuronales *Transformers*, esto no ocurre.

La capa de entrada se denota por  $y^{[0]} = x \in \mathbb{R}^d$  y no se cuenta en la profundidad de la red. Denotamos por  $f_\ell$  la función de capa que mapea de la capa  $\ell$  a la capa  $\ell + 1$  mediante

$$f_\ell(y^{[\ell]}) = W^{[\ell]}y^{[\ell]} + b^{[\ell]} \in \mathbb{R}^{d_{\ell+1}}, \quad \text{para } \ell = 0, \dots, L-1,$$

donde  $W^{[\ell]} \in \mathbb{R}^{d_{\ell+1} \times d_\ell}$  y  $b^{[\ell]} \in \mathbb{R}^{d_{\ell+1}}$ .

Para introducir no linealidad, componemos con una función de activación  $\sigma : \mathbb{R} \rightarrow \mathbb{R}$  que actúa componente a componente, es decir,

$$\sigma(y_1, \dots, y_n)^T = (\sigma(y_1), \dots, \sigma(y_n))^T.$$

La FNN puede representarse entonces como

$$y^{[\ell]} = f_\ell(y^{[\ell-1]}) = \sigma^{[\ell]}(W^{[\ell-1]}y^{[\ell-1]} + b^{[\ell-1]}), \quad \text{para } \ell = 1, \dots, L,$$

donde la función de activación  $\sigma^{[\ell]}$  puede variar de una capa a otra. Llamamos a  $y^{[\ell]}$  el vector de características de la capa  $\ell$ . De forma compacta, podemos escribir una FNN como una composición de sus funciones de capa:

$$y^{[L]} = f(u) = f_L \circ f_{L-1} \circ \cdots \circ f_1(u).$$

Expresado en forma matricial, las salidas de la capa  $k$  serán las siguientes:

$$f^{(k)} = \sigma \left( W^{(k)} f^{(k-1)} + b^{(k)} \right)$$

$$= \sigma \left( \begin{pmatrix} w_{11}^{(k)} & w_{12}^{(k)} & \cdots & w_{1n_{k-1}}^{(k)} \\ w_{21}^{(k)} & w_{22}^{(k)} & \cdots & w_{2n_{k-1}}^{(k)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n_k 1}^{(k)} & w_{n_k 2}^{(k)} & \cdots & w_{n_k n_{k-1}}^{(k)} \end{pmatrix} \begin{pmatrix} f_1^{(k-1)} \\ f_2^{(k-1)} \\ \vdots \\ f_{n_{k-1}}^{(k-1)} \end{pmatrix} + \begin{pmatrix} b_1^{(k)} \\ b_2^{(k)} \\ \vdots \\ b_{n_k}^{(k)} \end{pmatrix} \right)$$

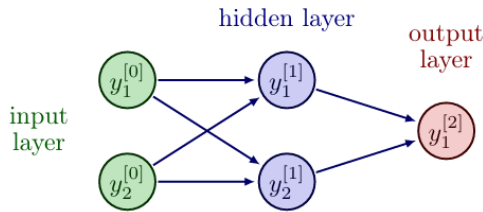
Esta formulación refuerza la elección de funciones de activación no lineales  $\sigma$ . En caso contrario, la salida  $y^{[L]}$  dependería linealmente de la entrada  $u$  y las capas ocultas podrían eliminarse, por ejemplo, con funciones de activación lineales, resolver tareas simples como modelar la función lógica “XOR” no sería posible. Sin embargo, en algunos casos puede ser conveniente tener una última capa lineal.

**Observación 2.** En la práctica, no es raro que la última capa de una FNN sea lineal, es decir,

$$y^{[L]} = W^{[L-1]} y^{[L-1]}.$$

Siempre que las capas anteriores sean no lineales, esto no reduce la capacidad expresiva de la red y suele utilizarse para alcanzar la dimensión de salida deseada. En este caso,  $W^{[L-1]}$  puede interpretarse como una reformulación lineal.

**Ejemplo.** Intentemos representar la función lógica “XOR” mediante una FNN con una capa oculta.



**Figura 1.4:** Una red neuronal prealimentada de 2 dimensiones en el input y una capa oculta con 2 nodos, i.e  $n_0 = n_1 = 2, n_2 = 1$  and  $L = 2$

Los parámetros

$$W^{[0]} = \begin{pmatrix} 1 & 1 \\ -1 & -1 \end{pmatrix}, \quad b^{[0]} = \begin{pmatrix} -1 \\ 1 \end{pmatrix}, \quad W^{[1]} = (1 \quad 1), \quad b^{[1]} = -2,$$

resuelven el problema y nos proveen la correspondiente tabla de verdad.

| input $y_1^0$ | input $y_2^0$ | $y_1^1$ | $y_2^1$ | $y_1^2$ (XOR) |
|---------------|---------------|---------|---------|---------------|
| 0             | 0             | 0       | 1       | 0             |
| 1             | 0             | 1       | 1       | 1             |
| 0             | 1             | 1       | 1       | 1             |
| 1             | 1             | 1       | 0       | 0             |

**Tabla 1.2:** Tabla de la operación XOR mediante una red con una capa oculta

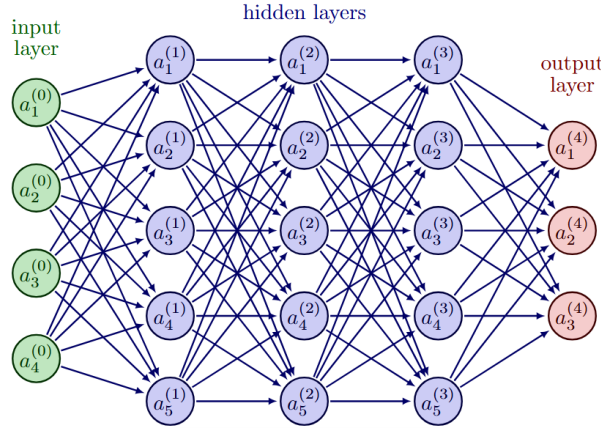
Para una red FNN el problema de optimización para múltiples capas ocultas puede expresarse mediante la notación anterior:

$$\min_{\{W^{[\ell]}\}_\ell, \{b^{[\ell]}\}_\ell} \mathcal{L}\left(\{y^{[L](i)}\}_i, \{u^{(i)}\}_i, \{W^{[\ell]}\}_\ell, \{b^{[\ell]}\}_\ell\right)$$

siendo  $y^{[L](i)} = f\left(u^{(i)}, \{W^{[\ell]}\}_\ell, \{b^{[\ell]}\}_\ell\right).$

Si agrupamos de nuevo todas las variables en un único vector  $\theta$ , este tendrá la siguiente dimensión:

$$\underbrace{n_0 n_1 + \cdots + n_{L-1} n_L}_{\text{pesos}} + \underbrace{n_1 + \cdots + n_L}_{\text{sesgos}}.$$



**Figura 1.5:** Una red neuronal prealimentada con 3 capas ocultas, con tamaños de capa  $n_0 = 3$ ,  $n_1 = n_2 = n_3 = 5$ ,  $n_4 = 1$  y profundidad  $L = 4$ . Al agrupar todas las variables de esta red en un vector, se obtiene  $\theta \in \mathbb{R}^{86}$ .

## Backpropagation

Cuando una red, por ejemplo una FNN, toma una entrada  $u$ , la pasa a través de sus capas y finalmente devuelve una salida  $y^{[L]}$ , la red propaga hacia adelante la información, a lo esto se le llama propagación hacia adelante. Seguidamente, se le asigna una pérdida a la salida y nuestro objetivo es emplear el gradiente de la función de pérdida  $\nabla \mathcal{L}(\theta)$  para actualizar las variables de la red  $\theta \in \mathbb{R}^K$ . En una FNN el número de parámetros (variables) será  $K = n_0 \cdot n_1 + \cdots + n_{L-1} \cdot n_L + n_1 + \cdots + n_L$ .

El gradiente está dado por

$$\nabla \mathcal{L}(\theta) = \begin{pmatrix} \frac{\partial \mathcal{L}(\theta)}{\partial \theta_1} \\ \vdots \\ \frac{\partial \mathcal{L}(\theta)}{\partial \theta_K} \end{pmatrix}.$$

Para una red neuronal feedforward (FNN) general de profundidad  $L$ , con múltiples nodos por capa, matrices de pesos  $W^{[\ell]}$  y vectores de sesgo  $b^{[\ell]}$ , el resultado general del gradiente, llamado retropropagación o *backpropagation*, se obtiene aplicando la regla de la cadena multivariable.

Primero se define formalmente cómo se propaga la información en cualquier capa  $\ell$  (donde  $\ell = 1, \dots, L$ ):

Pre-activación (Combinación lineal):

$$z^{[\ell]} = W^{[\ell-1]} y^{[\ell-1]} + b^{[\ell-1]} \in \mathbb{R}^{n_\ell}$$

Activación (Salida de la capa):

$$y^{[\ell]} = \sigma^{[\ell]}(z^{[\ell]}) \in \mathbb{R}^{n_\ell}$$

Donde  $n_\ell$  es el número de neuronas en la capa  $\ell$ , y  $y^{[0]} = u$  es el vector de entrada de la red.

Asumiendo que la función de pérdida  $\mathcal{L}$  toma únicamente la salida de la última capa  $y^{[L]}$  (como ocurre en el Error Cuadrático Medio), el gradiente para cualquier capa intermedia  $\ell = 0, \dots, L-1$  se calcula mediante las siguientes ecuaciones:

La derivada parcial de la función de pérdida con respecto a la matriz de pesos de la capa  $\ell$  es:

$$\frac{\partial \mathcal{L}}{\partial W^{[\ell]}} = \frac{\partial \mathcal{L}}{\partial y^{[L]}} \cdot \prod_{j=L}^{\ell+2} \left( \frac{\partial y^{[j]}}{\partial z^{[j]}} \cdot \frac{\partial z^{[j]}}{\partial y^{[j-1]}} \right) \cdot \frac{\partial y^{[\ell+1]}}{\partial z^{[\ell+1]}} \cdot \frac{\partial z^{[\ell+1]}}{\partial W^{[\ell]}}$$

La derivada parcial de la función de pérdida con respecto al vector de sesgos de la capa  $\ell$  es:

$$\frac{\partial \mathcal{L}}{\partial b^{[\ell]}} = \frac{\partial \mathcal{L}}{\partial y^{[L]}} \cdot \prod_{j=L}^{\ell+2} \left( \frac{\partial y^{[j]}}{\partial z^{[j]}} \cdot \frac{\partial z^{[j]}}{\partial y^{[j-1]}} \right) \cdot \frac{\partial y^{[\ell+1]}}{\partial z^{[\ell+1]}} \cdot \frac{\partial z^{[\ell+1]}}{\partial b^{[\ell]}}$$

Este método es computacionalmente muy eficiente debido a que está constantemente reutilizando las derivadas parciales que halla en cada iteración. Por ejemplo,  $\frac{\partial \mathcal{L}}{\partial y^{[L]}}$  es parte de todas las derivadas. Por lo tanto, si calculamos primero la derivada respecto a  $W^{[L-1]}$ , ya tenemos esta componente disponible y podremos reutilizarla en el cálculo de la derivada respecto a  $W^{[L-2]}$ , etc.

## Importancia de la arquitectura ResNet

Hemos visto que para una FNN con profundidad  $L$  tenemos la derivada

$$\frac{\partial \mathcal{L}}{\partial W^{[\ell]}} = \frac{\partial \mathcal{L}}{\partial y^{[L]}} \cdot \prod_{j=L}^{\ell+2} \left( \frac{\partial y^{[j]}}{\partial z^{[j]}} \cdot \frac{\partial z^{[j]}}{\partial y^{[j-1]}} \right) \cdot \frac{\partial y^{[\ell+1]}}{\partial z^{[\ell+1]}} \cdot \frac{\partial z^{[\ell+1]}}{\partial W^{[\ell]}}$$

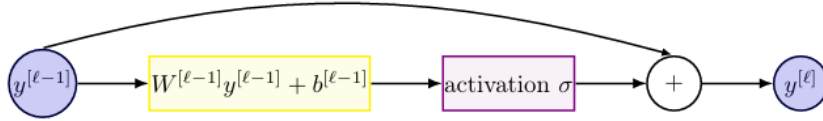
En el caso en que consideremos una red muy profunda, es decir, con un valor grande de  $L$ , el producto de derivadas parciales puede resultar problemático, especialmente si tomamos derivadas con respecto a variables de las primeras capas. Pueden ocurrir dos casos:

1. Si  $\|\frac{\partial y^{[j]}}{\partial y^{[j-1]}}\| < 1$  para todo  $j$ , el producto, y por lo tanto toda la derivada, tiende a cero para un  $L$  creciente. Este problema se conoce como **desvanecimiento del gradiente** (*vanishing gradient*).
2. Por otro lado, si  $\|\frac{\partial y^{[j]}}{\partial y^{[j-1]}}\| > 1$  para todo  $j$ , el producto, y por lo tanto toda la derivada, tiende a infinito para un  $L$  creciente. Este problema se conoce como **explosión del gradiente** (*exploding gradient*).

Las **Redes Residuales** (**ResNets**) han sido desarrolladas con la intención de resolver el problema del desvanecimiento del gradiente. Empleando la misma notación que en las FNNs, las capas simplificadas de una *ResNet* se pueden representar de la siguiente manera:

$$y^{[\ell]} = y^{[\ell-1]} + \sigma^{[\ell]} \left( W^{[\ell-1]} y^{[\ell-1]} + b^{[\ell-1]} \right) \quad \text{para } \ell = 1, \dots, L,$$

con  $y^{[0]} = u$  siendo los datos de entrada. Básicamente, una ResNet es una FNN con una **conexión de salto** (*skip connection*) añadida, es decir,  $+y^{[\ell-1]}$ , como se indica en la Figura 1.6.



**Figura 1.6:** Arquitectura de una ResNet. Observemos como en cada capa sumamos la anterior.

La diferencia es que en una red tradicional, las capas intentan aprender una función objetivo  $H(x)$  completamente nueva [12], partiendo desde cero. Al reformularlo como  $y = y^{[\ell-1]} + \sigma(\cdot)$ , obligamos a las capas a aprender únicamente la función residual  $F(x) = H(x) - x$ . Es decir, la red ya no tiene que reinventar la información en cada capa; solo tiene que aprender el “cambio”, el “residuo” que se le debe aplicar a la entrada anterior.

Las capas de la ResNet de esta forma solo funcionan si todos los vectores de características  $y^{[\ell]}$  tienen la misma dimensión  $n_\ell$ , de modo que podamos sumarlos. Para permitir tamaños de capa diferentes, necesitamos insertar operadores de proyección  $P_{\ell-1}^\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ , es decir:

$$y^{[\ell]} = P_{\ell-1}^\ell y^{[\ell-1]} + \sigma^{[\ell]} \left( W^{[\ell-1]} y^{[\ell-1]} + b^{[\ell-1]} \right) \quad \text{para } \ell = 1, \dots, L,$$

cuya única función es hacer que todos los vectores de características tengan la misma dimensión para poder ser sumados.

# Operadores Neuronales: aprendizaje en espacios de funciones

En esta sección, abordaremos la definición del objeto matemático más importante de este trabajo: **los Operadores Neuronales**.

Nuestro cometido será aproximar operadores sobre espacios de funciones (espacios de dimensión infinita). Para ello, vamos a valernos de las redes neuronales clásicas, que son mapeos entre espacios vectoriales de dimensión finita, combinadas con un operador integral adicional en cada capa. Con ello podremos extender el marco teórico de las redes neuronales al contexto de operadores entre espacios funcionales.

Las redes neuronales clásicas tratan de aproximar funciones

$$x \mapsto f(x)$$

entre espacios euclídeos de dimensión finita, como  $\mathbb{R}^d$ . Más específicamente, toman un *vector*  $x$  como entrada y devuelven otro vector  $f(x)$  como salida. En contraste, nosotros queremos construir aplicaciones

$$f \mapsto g$$

entre espacios (vectoriales) de funciones de dimensión infinita, como el espacio de funciones continuas, estas aplicaciones son conocidas como *operadores*. Los operadores toman una función  $f$  como entrada y devuelven otra función  $g$  como salida [3].

|                 | Neural Network                          | Neural Operator         | Input               | Output              |
|-----------------|-----------------------------------------|-------------------------|---------------------|---------------------|
| <b>Space</b>    | Euclidean spaces (e.g. $\mathbb{R}^d$ ) | Function spaces         | $\mathcal{F}$       | $\mathcal{G}$       |
| <b>Elements</b> | Vectors                                 | Functions               | $f \in \mathcal{F}$ | $g \in \mathcal{G}$ |
| <b>Domain</b>   | Integer indices                         | Real-valued coordinates | $x \in D_f$         | $y \in D_g$         |

Tabla 1.3: Notation and comparison of neural networks to neural operators.

El paso de un problema discreto (vectores) a uno continuo (funciones) requiere un cambio en como se procesan y manipulan los datos. Naturalmente, aunque nuestro interés esté en el aprendizaje de mapeos entre funciones, no solemos tener acceso a las funciones completas numéricamente. Por esto, los operadores neuronales están diseñados para trabajar en una discretización  $\{x_\ell, f(x_\ell)\}_{\ell=1}^L$  de la función  $f$ , asumiendo que observamos la función en un conjunto de puntos  $x_i$  (i.e., un punto nube o malla). Además, la salida del operador neuronal, que es otra función  $g$ , debe poder ser evaluada en cualquier punto  $y$  de su dominio.

## ¿Qué propiedades debe tener un operador neuronal?

Para aprender aplicaciones entre espacios de funciones, queríamos que se cumplieran las siguientes propiedades:

- **Agnóstico a la discretización:** el operador aprendido debe ser aplicable a cualquier discretización de la función de entrada y producir salidas consistentes para cualquier resolución.
- **Número fijo de parámetros:** el número de parámetros aprendibles necesita ser fijo e independiente de las discretizaciones.

- **Aproximación Universal:** el conjunto de todos los operadores neuronales debería ser capaz de aproximar, con un error arbitrario, cualquier operador suficientemente regular.

## De Redes Neuronales a Operadores: Construyendo las capas de los Operadores Neuronales

Una forma de diseñar redes que aproximen operadores consiste en construir capas simples que puedan componerse formando un modelo de dichos operadores. Cada capa debe ser en sí misma una aplicación entre espacios de funciones, de modo que las salidas intermedias después de cada capa sean *funciones* (a diferencia de los vectores en las redes neuronales estándar).

En este apartado, presentamos cómo las capas de una redes neuronales más populares, el perceptrón multicapa o MLP, pueden convertirse en capas de operadores neuronales, garantizando que se mantengan consistentes a través de las diferentes resoluciones.

Asumamos que se nos dan  $n$  valores  $f_i = f(x_i)$  de una función  $f$  en los puntos  $(x_i)_{i=1}^n$ . Con estos datos podríamos diseñar un modelo que mapee la listas

$$(f(x_1), f(x_2), \dots, f(x_n)) \mapsto (g(y_1), g(y_2), \dots, g(y_m))$$

mediante capa completamente conectada con una matriz de pesos  $\mathbf{K} \in \mathbb{R}^{m \times n}$  y un vector de sesgo  $\mathbf{b} \in \mathbb{R}^m$  [3]. La coordenada  $j$ -ésima de la salida de la capa puede escribirse entonces como:

$$g_j := (\mathbf{K}\mathbf{f} + \mathbf{b})_j = \sum_{i=1}^n K_{ji}f_i + b_j.$$

Pero en este caso los pesos se aprenden para unas dimensiones de entrada y salida  $n$  y  $m$  concretas que no permiten trabajar con diferentes discretizaciones.

En lugar de aprender directamente los pesos para un conjunto fijo de índices, podemos aprender una función que mapee los puntos de entrada  $x$  y de consulta  $y$  (*query points*) al valor de los pesos, ponderados por la medida de los puntos de entrada, así

$$g_j = \sum_i K_{ji}f_i + b_j \quad \text{se convierte en} \quad g(y_j) = \sum_i \kappa(x_i, y_j) f(x_i) \Delta_i + b(y_j).$$

**Transformación en un operador integral** Para hacer que esta operación sea agnóstica a la discretización, asumimos que  $K_{ji}$  y  $b_j$  son evaluaciones de las *funciones*  $\kappa$  y  $b$  en los puntos de entrada  $x_i$  y los puntos de consulta  $y_j$ . El resultado es un mapeo desde la función de entrada  $f$  hacia una función de salida  $g$  dada por:

$$g(y_j) := \sum_{i=1}^n \kappa(x_i, y_j) f(x_i) \Delta_i + b(y_j) \approx \int_{D_f} \kappa(x, y_j) f(x) dx + b(y_j) = \mathcal{K}(f)(y_j) + b(y_j)$$

donde formalmente definimos  $K_{ji} := \kappa(x_i, y_j) \Delta_i$  y  $b_j := b(y_j)$ .

Nótese que introducimos pesos de cuadratura  $\Delta_i$ , los cuales aseguran que la suma aproxime la representación integral y que la salida sea agnóstica a la resolución. Podemos ver la suma como una suma de Riemann para la integral, que converge a medida que refinamos la discretización, es decir, aumentamos  $n$ .

Para mallas unidimensionales, tomaremos  $\Delta_i = \frac{x_{i+1} - x_{i-1}}{2}$ . Para nubes de puntos más generales, podemos particionar el dominio  $D_f$  y tomar el volumen de la subregión que contiene a  $x_i$ .

Las funciones de núcleo (*kernel*)  $\kappa$  y de sesgo  $b$  se parametrizan mediante redes neuronales y se aprenden utilizando retropropagación (*backpropagation*).

De esta manera hemos convertido la operación de regresión lineal o capa completamente conectada en espacios de dimensión finita

$$x \mapsto Wx + b$$

en otra en espacios de dimensión infinita mediante el operador lineal afín

$$f \mapsto \mathcal{K}(f) + b$$

A diferencia del MLP, esta expresión ahora puede evaluarse para un número arbitrario de puntos de entrada y de salida,  $n$  y  $m$ . Esto extiende de manera natural la definición de una **capa completamente conectada** (aplicación lineal afín entre los vectores  $\mathbf{f}$  y  $\mathbf{g}$ ) a un **operador lineal afín** entre las funciones  $f$  y  $g$ , donde, a medida que aumentamos la resolución (es decir, refinamos la discretización), el modelo converge un único operador integral.

## Definición de los Operadores Neuronales

La aproximación del operador  $\mathcal{G}^\dagger$  que se desea aprender se obtiene mediante la composición sucesiva de estos operadores lineales afines junto con funciones de activación no lineales. Conceptualmente, es el analogo a una red neuronal feed-forward estandar en espacios de dimensión infinita:

$$\mathcal{G}_\theta := \sigma_T(\mathcal{K}_{T-1} + b_{T-1}) \circ \cdots \circ \sigma_1(\mathcal{K}_0 + b_0)$$

esto se combina con la arquitectura ResNet<sup>5</sup>, explicada en [arquitectura ResNet](#), que se consigue sumando la capa anterior, lo que le permite aprender el residuo o cambio necesario respecto a una función identidad, en lugar de aproximar un mapa complejo desde cero<sup>6</sup>.

**Observación 3.** Si  $d_{v_{t+1}} = d_{v_t}$ , inicializamos  $W_t \in \mathbb{R}^{d_{v_{t+1}} \times d_{v_t}}$  como la identidad para que actúe como una conexión de salto o *skip connection*. Si  $d_{v_{t+1}} \neq d_{v_t}$ ,  $W_t$  se comporta como una proyección para poder sumar  $W_t v_t$  con  $(\mathcal{K}_t v_t + b_t)$ .

Con estas ideas ya podemos definir la estructura de un operador neuronal. Según la regularidad exigida a las funciones de entrada y salida, los espacios con los que trataremos podrán ser de los siguientes tipos:

- Consideramos  $\mathcal{A}$  un espacio  $L^p$ , un espacio de Sobolev  $W^{m,p}$  (típico cuando trabajamos con ecuaciones en derivadas parciales), ver [Anexo](#), o un espacio de funciones continuas  $C^1$ . Todas ellas definidas sobre el dominio  $D$ .

<sup>5</sup>En la pagina 10 de [16], Kovachi habla de que se inspira en ResNet al poner  $W_t$ .

<sup>6</sup>Tambien se puede ver  $W$  como una suerte de convolución  $1 \times 1$  que combina y reduce los canales del dominio  $D_f$ , [7] [23].

- Consideramos  $\mathcal{U}$  un espacio  $L^p$ , un espacio de Sobolev  $W^{m,p}$ , o un espacio de funciones diferenciables  $C^m$ . Todas ellas definidas sobre el dominio  $D$ .

Tomemos  $a \in \mathcal{A}$  como las funciones de entrada definidas sobre un dominio acotado  $D \subset \mathbb{R}^n$  y valoradas en  $\mathbb{R}^{d_a}$  ( $a : D \rightarrow \mathbb{R}^{d_a}$ ). Y como funciones de salida  $u \in \mathcal{U}$  definidas sobre el dominio acotado  $D' \subset \mathbb{R}^{n'}$  y valoradas en  $\mathbb{R}^{d_u}$  ( $u : D' \rightarrow \mathbb{R}^{d_u}$ ). Sean  $D_t \subset \mathbb{R}^{d_t}$  dominios acotados. La arquitectura propuesta para  $\mathcal{G}_\theta : \mathcal{A} \rightarrow \mathcal{U}$  es la siguiente [16]:

**Definición 1.** Llamaremos **operador neuronal** a una aplicación

$$\mathcal{G}_\theta : \mathcal{A} \rightarrow \mathcal{U}$$

de la forma

$$\mathcal{G}_\theta := Q \circ \sigma_T(W_{T-1} + \mathcal{K}_{T-1} + b_{T-1}) \circ \cdots \circ \sigma_1(W_0 + \mathcal{K}_0 + b_0) \circ P,$$

donde para todo  $t = 0, \dots, T-1$ :

- $W_t \in \mathbb{R}^{d_{v_{t+1}} \times d_{v_t}}$  y  $b_t : D_{t+1} \rightarrow \mathbb{R}^{d_{v_{t+1}}}$  representan la **conexión de salto** y el **sesgo**.
- $\sigma_t : \mathbb{R}^{d_{v_{t+1}}} \rightarrow \mathbb{R}^{d_{v_{t+1}}}$  es la **función de activación** fijada.
- $P : \mathbb{R}^{d_a} \rightarrow \mathbb{R}^{d_{v_0}}$  (**elevación**) y  $Q : \mathbb{R}^{d_{v_T}} \rightarrow \mathbb{R}^{d_u}$  (**proyección**) actúan punto a punto:

$$(P(a))(x) = P(a(x)), \quad x \in D_0, \quad (Q(v_T))(x) = Q(v_T(x)), \quad x \in D_T.$$

Definimos  $v_0(x) := P(a(x))$  y  $u(x) := Q(v_T(x))$ . Normalmente  $d_a < d_{v_0}$  y  $d_{v_T} > d_u$ .

- $\mathcal{K}_t \in \mathcal{L}(L^2(D_t; \mathbb{R}^{d_{v_t}}), L^2(D_{t+1}; \mathbb{R}^{d_{v_{t+1}}}))$  es el **operador integral**

$$(\mathcal{K}_t(v_t))(x) = \int_{D_t} \kappa^{(t)}(x, y) \cdot v_t(y) d\mu_t(y), \quad \forall x \in D_{t+1},$$

con  $\kappa^{(t)}(x, y) \in C(D_{t+1} \times D_t, \mathbb{R}^{d_{v_{t+1}} \times d_{v_t}})$  y  $\mu_t$  una medida de Borel.

De modo que cada capa viene dada por

$$v_{t+1}(x) = \sigma_{t+1}\left(W_t v_t(x) + \int_{D_t} \kappa^{(t)}(x, y) \cdot v_t(y) d\mu_t(y) + b_t(x)\right), \quad \forall x \in D_{t+1}.$$

La diferencia entre la arquitectura propuesta y una red neuronal clásica es que todas las operaciones están definidas directamente en el espacio de funciones, y por tanto no dependen de ninguna discretización de los datos:

$$a : D_0 \rightarrow \mathbb{R}^{d_a} \xrightarrow{\mathcal{G}_\theta} u : D_T \rightarrow \mathbb{R}^{d_u}$$

es decir, el operador neuronal aproxima una aplicación entre espacios de funciones.

Las dimensiones de salida  $d_{v_0}, \dots, d_{v_T}$ , así como las dimensiones intermedias  $d_1, \dots, d_{T-1}$  y los dominios de definición  $D_1, \dots, D_{T-1}$ , son hiperparámetros de la arquitectura. Todas las funciones involucradas en esta arquitectura ( $k^{(t)}, b_t, W_t, P$  y  $Q$ ) se obtienen mediante aproximaciones por redes neuronales clásicas poco profundas. Todos ellos se agrupan en el vector de parámetros  $\theta \in \mathbb{R}^p$ .

Vamos a comentar las componentes de esta arquitectura una por una:

El **operador integral**  $\mathcal{K}_t$  es el análogo, en el espacio de funciones, de la matriz de pesos en una red feed-forward estándar, ya que son operadores lineales de dimensión infinita que transforman un espacio de funciones en otro.

Los **sesgos**  $b_t$ , que normalmente son vectores, se generalizan aquí como funciones. Además, siguiendo la intuición de la arquitectura ResNet [12], se añade un **conexión de salto**  $W_t$  que actúa sobre la salida de la capa anterior antes de aplicar la no linealidad.

La capa de **elevación**  $P$  (*lifting*) mapea el campo de entrada desde su dimensión de canal original  $C_{\text{in}}$  hacia una representación latente de mayor dimensión  $C_{\text{hidden}}$ , expandiendo así la capacidad de representación del modelo antes de aplicar toda la secuencia de capas. La capa de **proyección**  $Q$  realiza la operación inversa, mapeando la representación latente final a la dimensionalidad de salida deseada  $C_{\text{out}}$  [7].

Esta expansión enriquece el espacio de características internas al permitir que el modelo represente un conjunto más amplio de interacciones entre canales antes de aplicar la secuencia de capas de propia del modelo. Conceptualmente, esto se asemeja a cómo las redes neuronales convolucionales (CNNs) transforman una imagen RGB con 3 canales de color en una jerarquía de mapas de características con dimensionalidades de canal progresivamente mayores, tales como 64, 128 o 256 canales en las capas intermedias. Esto permite al modelo capturar relaciones espaciales y semánticas más ricas, que luego se comprimen de nuevo al espacio RGB original de 3 canales para la salida. En este caso,  $P$  expande la dimensión (aumenta el número de canales) de los puntos  $x_i$  de la malla.

La capa de elevación en los Operadores Neuronales expande la entrada en una representación latente de alta dimensión para capturar correlaciones complejas, mientras que la capa de proyección la reduce de nuevo a la dimensionalidad de salida deseada, asegurando que el modelo aprenda representaciones intermedias expresivas sin perder la correspondencia con las dimensiones de entrada y salida.

## Ventajas de los Operadores Neuronales

**Buena Formulación Matemática.** Una ventaja de los operadores neuronales radica en su fundamento matemático en la teoría de operadores. Los operadores neuronales están diseñados para aproximar estos operadores directamente en lugar de aprender aproximaciones discretizadas, a diferencia de otros métodos basados en redes neuronales como las CNNs. Esto asegura que el mapeo aprendido respete la naturaleza continua del problema, y quizás generalice mejor a nuevos entornos no vistos [7].

**Aproximación Universal de Operadores.** Los Operadores Neuronales son aproximadores universales de operadores, en el sentido de que cualquier operador suficientemente suave puede ser aproximado con una precisión arbitraria utilizando un Operador Neuronal. Nótese sin embargo, que al igual que con los teoremas de aproximación universal de funciones para redes neuronales, estos resultados son solo garantías teóricas. En la práctica, pueden surgir discrepancias debido a errores de discretización o entornos de

optimización complejos, lo que puede llevar a una precisión limitada.

**Representación de Funciones y Flexibilidad en la Discretización de la Salida.** Una ventaja de los operadores neuronales es su capacidad para producir funciones continuas que pueden ser consultadas en discretizaciones arbitrarias. Como resultado, los operadores neuronales se pueden evaluar a cualquier resolución sin necesidad de reentrenamiento. Esto es valioso cuando se necesitan predicciones de alta resolución pero los datos de entrenamiento están disponibles solo en resoluciones más bajas. Las predicciones del operador neuronal también se mantienen consistentes a través de las resoluciones, ya que corresponden a evaluaciones de la misma función subyacente a diferentes resoluciones, eliminando así los artefactos que típicamente surgen de la interpolación de discreto a continuo. Adicionalmente, la forma funcional de la salida también se puede usar para calcular derivadas, integrales, etc, evitando las limitaciones de precisión que surgen de hacer que la salida del modelo tenga resolución fija. También previene el sobreajuste del operador a una resolución concreta y permite predecir en resoluciones arbitrarias debido a que es agnóstica a la discretización.

**Invarianza de Resolución y Convergencia.** Los operadores neuronales poseen la capacidad de ser evaluados a resoluciones arbitrarias, ofreciendo varias ventajas importantes para la computación científica. Poseen invarianza a la discretización, esto es, a medida que la resolución de entrada aumenta, las predicciones del operador neuronal convergen hacia una función de solución continua. Esta propiedad, asegura que el operador aprendido mantenga las características físicas del problema a lo largo de las escalas.

**Eficiencia de Datos y Ventajas de Entrenamiento.** Los operadores neuronales ofrecen la capacidad de aprender a partir de conjuntos de datos que abarcan múltiples resoluciones, proporcionando ventajas sustanciales tanto en eficiencia como en generalización. Esta propiedad permite a los modelos integrar datos muestreados a diferentes resoluciones espaciales o temporales, maximizando el uso de los recursos computacionales disponibles y los datos experimentales. Además, el aumento de datos (*data augmentation*), que es una técnica utilizada en aprendizaje automático para generar nuevas muestras de entrenamiento a partir de datos ya existentes, se puede lograr incorporando datos de múltiples resoluciones del mismo sistema físico, enriqueciendo el tamaño y la diversidad del conjunto de entrenamiento.

**Velocidad de Inferencia y Diferenciabilidad.** Una vez entrenados, los operadores neuronales pueden generar salidas de alta resolución a una fracción del costo computacional de los solucionadores numéricos tradicionales, siendo entre cien hasta un millón de veces más rápidos. Estas capacidades hacen de los operadores neuronales una alternativa eficiente y escalable para simulaciones en sistemas físicos complejos. Además, la mayoría de las arquitecturas de operadores neuronales son diferenciables. Su capacidad para generar predicciones fiables en milisegundos permite una exploración exhaustiva de grandes espacios de parámetros que, de otro modo, sería computacionalmente prohibitiva utilizando solucionadores tradicionales. Al mismo tiempo, la naturaleza diferenciable de los operadores neuronales permite utilizar métodos de optimización como el descenso de gradiente o el *backpropagation*.

**Beneficios de Implementación Práctica.** Al operar directamente en espacios de funciones en lugar de en cuadrículas densas, logran una eficiencia de memoria notable, lo que permite procesar entradas y salidas de alta resolución sin incurrir en altos costos de almacenamiento. Su estructura basada en redes neuronales permite la paralelización, lo que proporciona un cómputo eficiente y facilita el despliegue en hardware moderno.

## Aproximación Universal de Operadores Neuronales

Nos centraremos en el estudio de la densidad de la clase de los operadores neuronales como aproximadores de operadores del tipo  $G^\dagger : \mathcal{A} \rightarrow \mathcal{U}$ , donde  $\mathcal{A}$  y  $\mathcal{U}$  son espacios de Banach de funciones definidos sobre dominios Lipschitz. Estos espacios pueden ser espacios  $L^p$ , espacios de Sobolev  $W^{m,p}$  o espacios de funciones continuas y diferenciables  $C^m$ , según la regularidad exigida a las funciones de entrada y salida.

Para ello se consideran dos nociones de aproximación. La primera mide el error medio respecto a una medida de probabilidad  $\mu$  sobre el espacio de entrada, mediante la norma de Bochner  $L_\mu^2(\mathcal{A}; \mathcal{U})$ . La segunda mide el error de forma uniforme sobre subconjuntos compactos  $K \subset \mathcal{A}$ , mediante la norma del supremo.

**Motivaciones por las que se estudian ambas topologías.** Supongamos que queremos estudiar el operador  $\mathcal{G}^\dagger : \mathcal{A} \rightarrow \mathcal{U}$ , y necesitamos comprobar si el operador neuronal  $\mathcal{G}_\theta$  lo aproxima bien.

Imaginemos que  $\mathcal{G}_\theta$  tiene un error promedio (norma de Bochner) con respecto a  $\mathcal{G}^\dagger$  muy pequeño, esto es

$$\|\mathcal{G}^\dagger - \mathcal{G}_\theta\|_{L_\mu^2(\mathcal{A}; \mathcal{U})}^2 = \int_{\mathcal{A}} \|\mathcal{G}^\dagger(a) - \mathcal{G}_\theta(a)\|_{\mathcal{U}}^2 d\mu(a) \approx 0.$$

Pero puede pasar que para ciertas entradas  $a \in \mathcal{A}$  el error sea muy grande

$$\|\mathcal{G}^\dagger(a) - \mathcal{G}_\theta(a)\|_{\mathcal{U}} \gg 0$$

Este inconveniente se evita dentro de un compacto  $K$  imponiendo

$$\sup_{a \in K} \|\mathcal{G}^\dagger(a) - \mathcal{G}_\theta(a)\|_{\mathcal{U}} \approx 0 \quad \text{con } K \subset \mathcal{A} \text{ un compacto}$$

De esta manera el error queda uniformemente controlado para todas las funciones pertenecientes al compacto  $K$ .

- **La Topología de Bochner (Norma  $L_\mu^2$ ):** Mide la capacidad del operador neuronal para aproximar globalmente a  $\mathcal{G}^\dagger$  respecto a la distribución  $\mu$ .
- **La Topología Uniforme (Norma  $\sup_{a \in K}$ ):** Nos garantiza un control uniforme del error dentro de un compacto  $K$ , evitando que existan diferencias muy grandes entre las entradas.

Primero veremos que los operadores neuronales son densos en los operadores continuos  $G^\dagger : \mathcal{A} \rightarrow \mathcal{U}$  en la topología de convergencia uniforme en compactos [16]:

**Teorema 1.** *Supóngase que  $\mathcal{G}^\dagger : \mathcal{A} \rightarrow \mathcal{U}$  es continua y sean  $\sigma_1 \in A_L^0$ ,  $\sigma_2 \in A_0$ , y  $\sigma_3 \in A_{m_2}$ , ver [Anexo](#).*

*Entonces, para todo conjunto compacto  $K \subset \mathcal{A}$  y  $0 < \varepsilon \leq 1$ , existe un número  $N \in \mathbb{N}$  y un operador neuronal  $\mathcal{G} \in \mathcal{NO}_N(\sigma_1, \sigma_2, \sigma_3; D, D')$  tal que*

$$\sup_{a \in K} \|\mathcal{G}^\dagger(a) - \mathcal{G}(a)\|_{\mathcal{U}} \leq \varepsilon.$$

*Además, si  $\mathcal{U}$  es un espacio de Hilbert y  $\sigma_1 \in BA$  y, para algún  $M > 0$ , tenemos que  $\|\mathcal{G}^\dagger(a)\|_{\mathcal{U}} \leq M$  para todo  $a \in \mathcal{A}$  entonces este  $\mathcal{G}$  puede elegirse de modo que*

$$\|\mathcal{G}(a)\|_{\mathcal{U}} \leq 4M, \quad \forall a \in \mathcal{A}.$$

*Demostración.* Seguimos la demostración de Kovachki et al. adaptando la notación a la empleada en este trabajo. En la prueba se utilizan resultados auxiliares desarrollados en dicha referencia, en particular la aproximación de aplicaciones lineales mediante operadores integrales y la aproximación de redes neuronales clásicas mediante operadores neuronales.

Sea  $\epsilon > 0$ . En primer lugar, podemos aproximar el operador continuo  $\mathcal{G}^\dagger$  sobre el compacto  $K$  mediante un operador de la forma

$$\mathcal{G}_1 = G \circ \psi \circ F : \mathcal{A} \rightarrow \mathcal{U}$$

tal que

$$\sup_{a \in K} \|\mathcal{G}^\dagger(a) - \mathcal{G}_1(a)\|_{\mathcal{U}} \leq \frac{\epsilon}{2}.$$

donde  $F : \mathcal{A} \rightarrow \mathbb{R}^J$  y  $G : \mathbb{R}^{J'} \rightarrow \mathcal{U}$  son aplicaciones lineales continuas y  $\psi \in C(\mathbb{R}^J; \mathbb{R}^{J'})$  para ciertos  $J, J' \in \mathbb{N}$ .

A continuación, podemos aproximar  $F$  mediante una sucesión de operadores integrales  $F_t$ . Por tanto, existe una sucesión

$$F_t \in \text{IO}(\sigma_2; D, \mathbb{R}, \mathbb{R}^J), \quad t = 1, 2, \dots,$$

tal que

$$\sup_{a \in K} \sup_{x \in \overline{D}} |(F_t(a))(x) - F(a)|_1 \leq \frac{1}{t}.$$

En particular, cada  $F_t(a)$  es constante respecto de la variable  $x$ , de modo que podemos identificar su rango con un subconjunto de  $\mathbb{R}^J$ . Definimos

$$Z := \bigcup_{t=1}^{\infty} F_t(K) \cup F(K) \subset \mathbb{R}^J.$$

Este conjunto es compacto. Como  $\psi$  es continua, entonces es uniformemente continua sobre  $Z$ . Por tanto, existe una aplicación

$$\omega : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$$

continua, no negativa y no decreciente, con  $\omega(s) \rightarrow \omega(0) = 0$  cuando  $s \rightarrow 0$ , tal que

$$|\psi(z_1) - \psi(z_2)|_1 \leq \omega(|z_1 - z_2|_1), \quad \forall z_1, z_2 \in Z.$$

Escogemos entonces  $T \in \mathbb{N}$  suficientemente grande para que

$$\sup_{a \in K} \omega(|F(a) - F_T(a)|_1) \leq \frac{\epsilon}{6\|G\|}.$$

Como  $F_T(K)$  es compacto y  $\psi$  es continua, podemos aproximar  $\psi$  uniformemente sobre  $F_T(K)$  mediante una red neuronal clásica<sup>7</sup>. Es decir, existe una red neuronal  $\psi_\theta : \mathbb{R}^J \rightarrow \mathbb{R}^{J'}$  tal que

$$\sup_{q \in F_T(K)} |\psi(q) - \psi_\theta(q)|_1 \leq \frac{\epsilon}{6\|G\|}.$$

---

<sup>7</sup>Teorema de Aproximación Universal para Redes Neuronales Clásicas.

Además, los operadores neuronales pueden reproducir la acción de una red neuronal clásica cuando actúan sobre funciones constantes. Por tanto, existen operadores

$$S_1 \in \text{IO} \left( \sigma_1; D, \mathbb{R}^J, \mathbb{R}^{d_1} \right), \dots, S_{N-1} \in \text{IO} \left( \sigma_1; D, \mathbb{R}^{d_{N-1}}, \mathbb{R}^{J'} \right)$$

para algún  $N \in \mathbb{N}_{\geq 2}$  y ciertos  $d_1, \dots, d_{N-1} \in \mathbb{N}$ , tales que, definiendo

$$\tilde{\psi}(f) := (S_{N-1} \circ \sigma_1 \circ \dots \circ S_2 \circ \sigma_1 \circ S_1)(f),$$

se cumple

$$\sup_{q \in F_T(K)} \sup_{x \in \overline{D}} |\psi(q) - \tilde{\psi}(q\mathbf{1})(x)|_1 \leq \frac{\epsilon}{6\|G\|}.$$

siendo  $\mathbf{1}(x)$  la función constante  $\mathbf{1}$ . Por construcción,  $\tilde{\psi}$  transforma funciones constantes en funciones constantes, por lo que, sobre este tipo de entradas, puede identificarse con una aplicación de  $\mathbb{R}^J$  en  $\mathbb{R}^{J'}$ . Entonces

$$(\tilde{\psi} \circ F_T)(K) \subset \mathbb{R}^{J'} \quad \text{es compacto.}$$

El siguiente paso consiste en aproximar el operador lineal  $G$  mediante un operador integral cuyo núcleo está parametrizado por una red neuronal. Así, existe una red neuronal

$$\kappa \in N_L \left( \sigma_3; \mathbb{R}^{d'} \times \mathbb{R}^{d'}, \mathbb{R}^{1 \times J'} \right)$$

para algún  $L \in \mathbb{N}$ , tal que el operador

$$\tilde{G}(f) := \int_{D'} \kappa(\cdot, y) f(y) \, dy$$

satisface

$$\sup_{y \in (\tilde{\psi} \circ F_T)(K)} \|G(y) - \tilde{G}(y\mathbf{1})\|_{\mathcal{U}} \leq \frac{\epsilon}{6}.$$

Definimos ahora el operador de aproximación

$$\mathcal{G}(a) := \left( \tilde{G} \circ \tilde{\psi} \circ F_T \right)(a) = \int_{D'} \kappa(\cdot, y) \left( (S_{N-1} \circ \sigma_1 \circ \dots \circ \sigma_1 \circ S_1 \circ F_T)(a) \right)(y) \, dy, \quad a \in \mathcal{A}.$$

Por construcción,  $\mathcal{G}$  pertenece a la clase de operadores neuronales  $\text{NO}_N(\sigma_1, \sigma_2, \sigma_3; D, D')$ .

Veamos que  $\mathcal{G}$  aproxima a  $\mathcal{G}^\dagger$  sobre  $K$ . Para  $a \in K$ , definimos

$$a_1 := (\psi \circ F)(a), \quad \tilde{a}_1 := (\tilde{\psi} \circ F_T)(a).$$

Entonces

$$\mathcal{G}_1(a) = G(a_1), \quad \mathcal{G}(a) = \tilde{G}(\tilde{a}_1).$$

Por la desigualdad triangular,

$$\begin{aligned} \|\mathcal{G}_1(a) - \mathcal{G}(a)\|_{\mathcal{U}} &\leq \|G(a_1) - G(\tilde{a}_1)\|_{\mathcal{U}} + \|G(\tilde{a}_1) - \tilde{G}(\tilde{a}_1)\|_{\mathcal{U}} \\ &\leq \|G\| |a_1 - \tilde{a}_1|_1 + \sup_{y \in (\tilde{\psi} \circ F_T)(K)} \|G(y) - \tilde{G}(y\mathbf{1})\|_{\mathcal{U}} \end{aligned}$$

$$\begin{aligned}
&\leq \frac{\epsilon}{6} + \|G\| \left| (\psi \circ F)(a) - (\psi \circ F_T)(a) \right|_1 + \|G\| \left| (\psi \circ F_T)(a) - (\tilde{\psi} \circ F_T)(a) \right|_1 \\
&\leq \frac{\epsilon}{6} + \|G\| \omega \left( |F(a) - F_T(a)|_1 \right) + \|G\| \sup_{q \in F_T(K)} |\psi(q) - \tilde{\psi}(q)|_1 \leq \frac{\epsilon}{2}.
\end{aligned}$$

Por tanto,

$$\|\mathcal{G}^\dagger(a) - \mathcal{G}(a)\|_{\mathcal{U}} \leq \|\mathcal{G}^\dagger(a) - \mathcal{G}_1(a)\|_{\mathcal{U}} + \|\mathcal{G}_1(a) - \mathcal{G}(a)\|_{\mathcal{U}} \leq \frac{\epsilon}{2} + \frac{\epsilon}{2} = \epsilon.$$

Como esto vale para todo  $a \in K$ , se concluye que

$$\sup_{a \in K} \|\mathcal{G}^\dagger(a) - \mathcal{G}(a)\|_{\mathcal{U}} \leq \epsilon.$$

Vayamos ahora con la demostración de la acotación  $\|\mathcal{G}(a)\|_{\mathcal{U}} \leq 4M$ . Sea

$$M := \sup_{a \in K} \|\mathcal{G}^\dagger(a)\|_{\mathcal{U}},$$

y supongamos, sin pérdida de generalidad, que  $M \geq 1$ . De la estimación anterior,

$$\|\mathcal{G}(a)\|_{\mathcal{U}} \leq \|\mathcal{G}(a) - \mathcal{G}^\dagger(a)\|_{\mathcal{U}} + \|\mathcal{G}^\dagger(a)\|_{\mathcal{U}} \leq \epsilon + M \leq 2M, \quad \forall a \in K.$$

Por la forma de la construcción, podemos escribir

$$\mathcal{G}(a) = \sum_{j=1}^{J'} \tilde{\psi}_j(F_T(a)) \varphi_j, \quad a \in \mathcal{A},$$

para alguna red neuronal  $\varphi : \mathbb{R}^{d'} \rightarrow \mathbb{R}^{J'}$ . Como  $\mathcal{U}$  es un espacio de Hilbert, podemos suponer que las funciones  $\varphi_j$  están ortonormalizadas. Entonces

$$|\tilde{\psi}(F_T(a))|_2 = \|\mathcal{G}(a)\|_{\mathcal{U}} \leq 2M, \quad \forall a \in K.$$

Definimos

$$W := (\tilde{\psi} \circ F_T)(K) \subset \mathbb{R}^{J'}.$$

Este conjunto es compacto y satisface

$$\text{diam}_2(W) = \sup_{x,y \in W} |x - y|_2 \leq \sup_{x,y \in W} (|x|_2 + |y|_2) \leq 4M.$$

Como  $\sigma_1 \in BA$ , existe una red neuronal  $\beta \in \mathcal{N}_R(\sigma_1; \mathbb{R}^{J'}, \mathbb{R}^{J'})$  para algún  $R \in \mathbb{N}$ , tal que

$$|\beta(x) - x|_2 \leq \epsilon, \quad \forall x \in W,$$

y, además,

$$|\beta(x)|_2 \leq 4M, \quad \forall x \in \mathbb{R}^{J'}.$$

Definimos entonces

$$\widehat{\mathcal{G}}(a) := \sum_{j=1}^{J'} \beta_j \left( \widetilde{\psi}(F_T(a)) \right) \varphi_j, \quad a \in \mathcal{A}.$$

Por construcción,  $\widehat{\mathcal{G}}$  sigue perteneciendo a la clase de operadores neuronales considerada. Además,

$$\sup_{a \in K} \|\mathcal{G}(a) - \widehat{\mathcal{G}}(a)\|_{\mathcal{U}} \leq \sup_{w \in W} |w - \beta(w)|_2 \leq \epsilon.$$

Por tanto,  $\widehat{\mathcal{G}}$  conserva la aproximación sobre  $K$ .

Además,

$$\|\widetilde{\mathcal{G}}(a)\|_{\mathcal{U}} \leq \|\widetilde{\mathcal{G}}(a) - \mathcal{G}(a)\|_{\mathcal{U}} + \|\mathcal{G}(a)\|_{\mathcal{U}} \leq \epsilon + 2M \leq 3M, \quad \forall a \in K.$$

Sea  $a \in \mathcal{A} \setminus K$  entonces existe  $q \in \mathbb{R}^{J'} \setminus W$  tal que  $\widetilde{\psi}(F_T(a)) = q$  y

$$\|\widetilde{\mathcal{G}}(a)\|_{\mathcal{U}} = |\beta(q)|_2 \leq 4M$$

como se quería demostrar. ■

Aunque la densidad en el espacio de operadores continuos sea un resultado fundamental desde el punto de vista teórico, en la práctica los modelos de aprendizaje se entrenan minimizando errores cuadráticos medios sobre una distribución de datos. Por ello es importante estudiar la densidad de los operadores neuronales en espacios de la forma  $L_\mu^2(\mathcal{A}; \mathcal{U})$ , donde  $\mu$  es una medida de probabilidad sobre el espacio de entrada  $\mathcal{A}$ .

El siguiente teorema muestra que los operadores neuronales conservan sus propiedades de aproximación en este contexto probabilístico[16], siendo densos en  $L_\mu^2(\mathcal{A}; \mathcal{U})$  cuando  $\mathcal{U}$  es un espacio de Hilbert separable.

**Teorema 2.** Sea  $D' \subset \mathbb{R}^d$  un dominio de Lipschitz, y  $m_2 \in \mathbb{N}_0$ . Sea  $\mu$  una medida de probabilidad sobre  $\mathcal{A}$  y supóngase que  $\mathcal{G}^\dagger : \mathcal{A} \rightarrow H^{m_2}(D)$  es  $\mu$ -medible y  $\mathcal{G}^\dagger \in L_\mu^2(\mathcal{A}; H^{m_2}(D))$ . Sean  $\sigma_1 \in A_L^0 \cap BA$ ,  $\sigma_2 \in A_0$ , y  $\sigma_3 \in A_{m_2}$ .

Entonces, para todo  $0 < \varepsilon \leq 1$ , existe un número  $N \in \mathbb{N}$  y un operador neuronal  $\mathcal{G} \in NO_N(\sigma_1, \sigma_2, \sigma_3; D, D')$  tal que

$$\|\mathcal{G}^\dagger - \mathcal{G}\|_{L_\mu^2(\mathcal{A}; H^{m_2}(D))} \leq \varepsilon.$$

*Demostración.* Sea  $\mathcal{U} = H^{m_2}(D)$ . Para cualquier  $R > 0$ , definimos

$$\mathcal{G}_R^\dagger(a) := \begin{cases} \mathcal{G}^\dagger(a), & \|\mathcal{G}^\dagger(a)\|_{\mathcal{U}} \leq R, \\ \frac{R}{\|\mathcal{G}^\dagger(a)\|_{\mathcal{U}}} \mathcal{G}^\dagger(a), & \text{en otro caso,} \end{cases} \quad \text{para todo } a \in \mathcal{A}.$$

Como  $\mathcal{G}_R^\dagger \rightarrow \mathcal{G}^\dagger$  para casi en todo punto medida  $\mu$  cuando  $R \rightarrow \infty$ ,  $\mathcal{G}^\dagger \in L_\mu^2(\mathcal{A}; \mathcal{U})$ , y claramente  $\|\mathcal{G}_R^\dagger(a)\|_{\mathcal{U}} \leq \|\mathcal{G}^\dagger(a)\|_{\mathcal{U}}$  para todo  $a \in \mathcal{A}$ , podemos aplicar el teorema de convergencia dominada para integrales de Bochner y encontrar  $R > 0$  suficientemente grande tal que

$$\|\mathcal{G}_R^\dagger - \mathcal{G}^\dagger\|_{L_\mu^2(\mathcal{A}; \mathcal{U})} \leq \frac{\varepsilon}{3}.$$

Aplicando el teorema de Lusin [19] podemos encontrar un conjunto compacto  $K \subset \mathcal{A}$  tal que

$$\mu(\mathcal{A} \setminus K) \leq \frac{\varepsilon^2}{153R^2}$$

y  $\mathcal{G}_R^\dagger|_K$  es continua. Como  $K$  es cerrado, podemos extender a una aplicación continua  $\tilde{\mathcal{G}}_R^\dagger : \mathcal{A} \rightarrow \mathcal{U}$  tal que  $\tilde{\mathcal{G}}_R^\dagger(a) = \mathcal{G}_R^\dagger(a)$  para todo  $a \in K$  y

$$\sup_{a \in \mathcal{A}} \|\tilde{\mathcal{G}}_R^\dagger(a)\| \leq \sup_{a \in \mathcal{A}} \|\mathcal{G}_R^\dagger(a)\| \leq R.$$

Aplicando el **Teorema 1.** a  $\tilde{\mathcal{G}}_R^\dagger$ , obtenemos que existe un número  $N \in \mathbb{N}$  y un operador neuronal  $\mathcal{G} \in NO_N(\sigma_1, \sigma_2, \sigma_3; D, D')$  tal que

$$\sup_{a \in K} \|\mathcal{G}(a) - \mathcal{G}_R^\dagger(a)\|_{\mathcal{U}} \leq \frac{\sqrt{2\varepsilon}}{3}.$$

y

$$\sup_{a \in \mathcal{A}} \|\mathcal{G}(a)\|_{\mathcal{U}} \leq 4R.$$

Entonces tenemos

$$\begin{aligned} \|\mathcal{G}^\dagger - \mathcal{G}\|_{L_\mu^2(\mathcal{A}; \mathcal{U})} &\leq \|\mathcal{G}^\dagger - \mathcal{G}_R^\dagger\|_{L_\mu^2(\mathcal{A}; \mathcal{U})} + \|\mathcal{G}_R^\dagger - \mathcal{G}\|_{L_\mu^2(\mathcal{A}; \mathcal{U})} \\ &\leq \frac{\varepsilon}{3} + \left( \int_K \|\mathcal{G}_R^\dagger(a) - \mathcal{G}(a)\|_{\mathcal{U}}^2 d\mu(a) + \int_{\mathcal{A} \setminus K} \|\mathcal{G}_R^\dagger(a) - \mathcal{G}(a)\|_{\mathcal{U}}^2 d\mu(a) \right)^{\frac{1}{2}} \\ &\leq \frac{\varepsilon}{3} + \left( \frac{2\varepsilon^2}{9} + 2 \left( \sup_{a \in \mathcal{A}} \|\mathcal{G}_R^\dagger(a)\|_{\mathcal{U}} + \|\mathcal{G}(a)\|_{\mathcal{U}} \right)^2 \cdot \mu(\mathcal{A} \setminus K) \right)^{\frac{1}{2}} \\ &\leq \frac{\varepsilon}{3} + \left( \frac{2\varepsilon^2}{9} + 34R^2\mu(\mathcal{A} \setminus K) \right)^{\frac{1}{2}} \leq \frac{\varepsilon}{3} + \left( \frac{4\varepsilon^2}{9} \right)^{\frac{1}{2}} = \varepsilon, \end{aligned}$$

como se quería demostrar. ■

## Invarianza respecto a la Discretización

Dado que nuestros datos de entrada y salida  $(a^{(i)}, u^{(i)})$  son funciones, para poder trabajar con ellos de computacionalmente, necesitamos restringirlas a un dominio discreto; esto es cambiar dichas funciones continuas por otras que sean “buenas representantes discretas” de estas (aclararemos ahora que significa esto). Así, tomando  $D \subset \mathbb{R}^n$  un dominio acotado:

**Definición 2.** Sea una cadena  $D_1 \subset D_2 \subset \dots \subset D_\ell \subset \dots \subset D$ , con  $|D_\ell| = \ell$  para cada  $\ell \in \mathbb{N}$ . Decimos que dicha cadena es un **refinamiento discreto de  $D$**  si para todo  $\varepsilon > 0$  existe  $\ell = \ell(\varepsilon)$  de manera que

$$D \subseteq \bigcup_{x \in D_\ell} \mathcal{B}_\varepsilon(x)$$

**Definición 3.** Sea  $\{D_\ell\}_{\ell=1}^\infty$  un refinamiento discreto del dominio  $D \subset \mathbb{R}^n$ . Decimos que  $D_\ell$  es una **discretización de  $D$** .

Notemos que dado  $a : D \subset \mathbb{R}^n \rightarrow \mathbb{R}^m$ , una evaluación puntual  $\{x_\ell, a(x_\ell)\}_{\ell=1}^L$  puede verse como un vector en  $\mathbb{R}^{\ell_n} \times \mathbb{R}^{\ell_m}$ .

Así, dada una discretización  $D^{(i)} = \{x_\ell^{(i)}\}_{\ell=1}^L \subset D$ , tenemos nuestras funciones discretizadas  $a^{(i)}|_{D^{(i)}}, u^{(i)}|_{D^{(i)}} \in \mathbb{R}^L$ . Además, la clase de operadores paramétricos es consistente, en el sentido de que, dado un conjunto fijo de parámetros, el refinamiento de la discretización de entrada converge hacia el operador neuronal verdadero en el espacio de funciones.

Nos referimos a las arquitecturas que poseen esta propiedad como arquitecturas del espacio de funciones invariantes respecto a la malla o arquitecturas **invariantes respecto a la discretización elegida**.

Sea  $\mathcal{A}$  un espacio de Banach de funciones  $D \rightarrow \mathbb{R}^m$ ,  $\mathcal{G} : \mathcal{A} \rightarrow \mathcal{U}$  un operador neuronal,  $D_\ell$  una discretización de  $D$  y  $\hat{\mathcal{G}} : \mathbb{R}^{\ell_n} \times \mathbb{R}^{\ell_m} \rightarrow \mathcal{U}$  una aplicación:

**Definición 4.** Sea  $K \subset \mathcal{A}$  un compacto, se define el **error debido a la discretización elegida** como

$$R_K(\mathcal{G}, \hat{\mathcal{G}}, D_\ell) = \sup_{a \in K} \left\| \hat{\mathcal{G}}\left(D_\ell, a|_{D_\ell}\right) - \mathcal{G}(a) \right\|_{\mathcal{U}}$$

Sea  $\Theta \subset \mathbb{R}^p$  un espacio de parámetros de dimensión finita y  $\mathcal{G} : \mathcal{A} \times \Theta \rightarrow \mathcal{U}$  una aplicación que representa la clase paramétrica de operadores neuronales con parámetros  $\theta \in \Theta$ .

**Definición 5.** Dado un refinamiento discreto  $\{D_\ell\}_{\ell=1}^\infty$  del dominio  $D \subset \mathbb{R}^n$  decimos que un operador  $\mathcal{G} : \mathcal{A} \rightarrow \mathcal{U}$  es **invariante respecto a la discretización** si existe una sucesión de aplicaciones  $\hat{\mathcal{G}}_1, \hat{\mathcal{G}}_2, \dots$  tal que  $\hat{\mathcal{G}}_\ell : \mathbb{R}^{\ell_n} \times \mathbb{R}^{\ell_m} \times \Theta \rightarrow \mathcal{U}$  y para cada  $\theta \in \Theta$  y cada conjunto compacto  $K \subset \mathcal{A}$

$$\lim_{\ell \rightarrow \infty} R_K(\mathcal{G}(\cdot, \theta), \hat{\mathcal{G}}_\ell(\cdot, \cdot, \theta), D_\ell) = 0$$

Un modelo es *invariante respecto a la discretización* si, al cambiar la malla, no cambia la parametrización  $\theta$  del operador. Es decir, una misma elección de parámetros  $\theta \in \Theta$  debe representar un operador continuo  $G(\cdot, \theta)$ , y sus versiones discretizadas  $\hat{G}_\ell(\cdot, \cdot, \theta)$  deben aproximarlo cada vez mejor al refinar la discretización. Por eso, la invarianza respecto a la discretización contiene dos ideas:

- **Independencia paramétrica de la malla:** los parámetros  $\theta$  no se redefinen al cambiar la resolución o la discretización utilizada.
- **Consistencia:** al refinar la discretización, las versiones discretas del modelo convergen hacia el operador continuo que representan.

**Teorema 3.** Sean  $D \subset \mathbb{R}^d$  y  $D' \subset \mathbb{R}^{d'}$  dos dominios para algunos  $d, d' \in \mathbb{N}$ . Sean  $\mathcal{A}$  y  $\mathcal{U}$  espacios de funciones de Banach reales sobre  $D$  y  $D'$  respectivamente. Supongamos que  $\mathcal{A}$  y  $\mathcal{U}$  se embeben continuamente en  $C(D)$  y  $C(D')$  respectivamente, y que  $\sigma_1, \sigma_2, \sigma_3 \in C(\mathbb{R})$ . Entonces, para todo  $\varepsilon > 0$ , el conjunto de operadores neuronales  $\text{NO}_n(\sigma_1, \sigma_2, \sigma_3; D, D')$  cuyos elementos se consideran como aplicaciones  $\mathcal{A} \rightarrow \mathcal{U}$  es **invariante respecto a la discretización**.

*Demostración.* Sin pérdida de generalidad, supondremos que  $D = D'$  y, por inmersión continua, que  $\mathcal{A} = \mathcal{U} = C(\bar{D})$ . Además, no hay pérdida de generalidad al considerar el resultado para una sola capa.

$$\text{NO} = \left\{ f \mapsto \sigma_1 \left( \int_D \kappa(\cdot, y) f(y) dy + b \right) : \kappa \in N_n(\sigma_2; \mathbb{R}^{2d} \times \mathbb{R}^d), b \in N_n(\sigma_3; \mathbb{R}^d), n_1, n_2 \in \mathbb{N} \right\}.$$

Sea  $K \subset \mathcal{A}$  un conjunto compacto y  $(D_j)_{j=1}^\infty$  un refinamiento discreto de  $D$ . A cada discretización  $D_j$  le asociamos las particiones  $P_1^{(j)}, \dots, P_j^{(j)} \subset D$ , disjuntas dos a dos, cada una contiene un único punto de  $D_j$ , tienen medida de Lebesgue positiva y

$$\prod_{k=1}^j P_k^{(j)} = D.$$

Para cualquier  $\mathcal{G} \in \text{NO}$  con parámetros  $\kappa$  y  $b$ , definimos la sucesión de aplicaciones  $\hat{\mathcal{G}}_j : \mathbb{R}^{jd} \times \mathbb{R}^d \rightarrow \mathcal{U}$  por

$$\hat{\mathcal{G}}_j(y_1, \dots, y_j, w_1, \dots, w_j) = \sigma_1 \left( \sum_{k=1}^j \kappa(\cdot, y_k) w_k |P_k^{(j)}| + b(\cdot) \right).$$

para todo  $y_k \in \mathbb{R}^d$  y  $w_k \in \mathbb{R}$ . Como  $K$  es compacto, existe una constante  $M > 0$  tal que

$$\sup_{a \in K} \|a\|_{\mathcal{U}} \leq M.$$

Por tanto,

$$\begin{aligned} \sup_{x \in \bar{D}} \sup_{j \in \mathbb{N}} \left| \int_D \kappa(x, y) a(y) dy + \sum_{k=1}^j \kappa(x, y_k) a(y_k) |P_k^{(j)}| + 2b(x) \right| \\ \leq 2 \left( M \cdot |D| \cdot \|\kappa\|_{C(\bar{D} \times \bar{D})} + \|b\|_{C(\bar{D})} \right) =: R. \end{aligned}$$

Por lo tanto, sólo necesitamos considerar  $\sigma_1$  como una aplicación  $[-R, R] \rightarrow \mathbb{R}$ . Así, por continuidad uniforme, existe una aplicación  $\omega : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$  creciente, continua y no negativa, con  $\omega(0) = 0$ , que satisface  $\omega(z) \rightarrow 0$  cuando  $z \rightarrow 0$  y

$$|\sigma_1(z_1) - \sigma_1(z_2)| \leq \omega(|z_1 - z_2|) \quad \forall z_1, z_2 \in [-R, R].$$

Sea  $\varepsilon > 0$ . Los resultados anteriores implican que, para demostrar que existe  $Q = Q(\varepsilon) \in \mathbb{N}$  tal que para todo  $m \geq Q$  se cumple

$$\sup_{a \in K} \|\hat{\mathcal{G}}_m(D_m, a|_{D_m}) - G(a)\|_{\mathcal{U}} < \varepsilon,$$

es suficiente demostrar que

$$\sup_{a \in K} \sup_{x \in \bar{D}} \left| \int_D \kappa(x, y) a(y) dy - \sum_{k=1}^m \kappa(x, y_k) a(y_k) |P_m^{(k)}| \right| < \varepsilon.$$

Para todo  $m \geq Q$ . Como  $K$  es compacto, podemos encontrar funciones  $a_1, \dots, a_N \in K$  tales que, para todo  $a \in K$ , existe algún  $n \in \{1, \dots, N\}$  con

$$\|a - a_n\|_{C(\bar{D})} \leq \frac{\varepsilon}{4 \cdot |D| \cdot \|\kappa\|_{C(\bar{D} \times \bar{D})}}.$$

Como  $(D_j)$  es un refinamiento discreto, por la convergencia de las sumas de Riemann, podemos encontrar  $q \in \mathbb{N}$  tal que, para todo  $t \geq q$ , se tiene

$$\sup_{x \in \bar{D}} \left| \sum_{k=1}^t \kappa(x, y_k) |P_t^{(k)}| - \int_D \kappa(x, y) dy \right| < |D| \cdot \|\kappa\|_{C(\bar{D} \times \bar{D})}.$$

donde  $D_t = \{y_1, \dots, y_t\}$ . De manera similar, podemos encontrar  $p_1, \dots, p_N \in \mathbb{N}$  tales que, para todo  $t_n \geq p_n$ , se cumple

$$\sup_{x \in \bar{D}} \left| \sum_{k=1}^{t_n} \kappa(x, y_k^{(n)}) a_n(y_k^{(n)}) |P_{t_n}^{(k)}| - \int_D \kappa(x, y) a_n(y) dy \right| < \frac{\varepsilon}{4}.$$

donde  $D_{t_n} = \{y_1^{(n)}, \dots, y_{t_n}^{(n)}\}$ . Sea  $m \geq \max\{q, p_1, \dots, p_N\}$  y denotemos  $D_m = \{y_1, \dots, y_m\}$ . Obsérvese que

$$\sup_{x \in \bar{D}} \left| \int_D \kappa(x, y) (a(y) - a_n(y)) dy \right| \leq |D| \cdot \|\kappa\|_{C(\bar{D} \times \bar{D})} \cdot \|a - a_n\|_{C(\bar{D})}.$$

Además,

$$\begin{aligned} \sup_{x \in \bar{D}} \left| \sum_{k=1}^m \kappa(x, y_k) (a_n(y_k) - a(y_k)) |P_m^{(k)}| \right| &\leq \|a_n - a\|_{C(\bar{D})} \sup_{x \in \bar{D}} \left| \sum_{k=1}^m \kappa(x, y_k) |P_m^{(k)}| \right| \\ &\leq \|a_n - a\|_{C(\bar{D})} \left( \sup_{x \in \bar{D}} \left| \sum_{k=1}^m \kappa(x, y_k) |P_m^{(k)}| - \int_D \kappa(x, y) dy \right| + \sup_{x \in \bar{D}} \left| \int_D \kappa(x, y) dy \right| \right) \\ &\leq 2|D| \cdot \|\kappa\|_{C(\bar{D} \times \bar{D})} \cdot \|a_n - a\|_{C(\bar{D})}. \end{aligned}$$

Por lo tanto, para todo  $a \in K$ , aplicando repetidamente la desigualdad triangular,

$$\begin{aligned} \sup_{x \in \bar{D}} \left| \int_D \kappa(x, y) a(y) dy - \sum_{k=1}^m \kappa(x, y_k) a(y_k) |P_m^{(k)}| \right| &\leq \\ &\leq \sup_{x \in \bar{D}} \left| \sum_{k=1}^m \kappa(x, y_k) a_n(y_k) |P_m^{(k)}| - \int_D \kappa(x, y) a_n(y) dy \right| + 3|D| \cdot \|\kappa\|_{C(\bar{D} \times \bar{D})} \|a - a_n\|_{C(\bar{D})} \end{aligned}$$

$$< \frac{\varepsilon}{4} + \frac{3\varepsilon}{4} = \varepsilon,$$

como se quería demostrar. ■

A la luz de estos dos últimos apartados, el error total de un operador neuronal puede descomponerse como:

$$\|\hat{G}_\theta(D_L, a|_{D_L}) - G^\dagger(a)\|_U \leq \underbrace{\|\hat{G}_\theta(D_L, a|_{D_L}) - G_\theta(a)\|_U}_{\text{error de discretización}} + \underbrace{\|G_\theta(a) - G^\dagger(a)\|_U}_{\text{error de aproximación}}.$$

Estos resultados muestran que el error de aproximación puede hacerse arbitrariamente pequeño ajustando  $\theta$ , y que el error de discretización puede hacerse arbitrariamente pequeño refinando la discretización (aumentando  $L$ ).

Por tanto, con un conjunto fijo de parámetros, un operador neuronal implementable en un ordenador puede aproximar operadores con precisión arbitraria.

## Comparativa con otros Métodos Numéricos

Compararemos los operadores neuronales con dos familias de métodos de resolución de ecuaciones diferenciales: los basados en Redes Neuronales Clásicas y los Métodos Numéricos Clásicos de resolución de EDPs [15].

### Operadores Neuronales frente a Redes Neuronales

En primer lugar, los basados en Redes Neuronales (como PINN o las redes convolucionales<sup>8</sup>) resuelven un problema de mapeo de vector a vector, es decir aprenden una función  $\mathbb{R}^n \rightarrow \mathbb{R}^m$  definida sobre espacios vectoriales de dimensión finita. Mientras que los operadores neuronales aprenden directamente el operador solución en un espacio de funciones de dimensión infinita  $\mathcal{A} \rightarrow \mathcal{U}$ , lo que aproxima mejor el problema real, pues sus salidas son funciones, no vectores.

Como se ha visto en apartados anteriores, los operadores neuronales poseen propiedades de aproximación universal para ciertas clases de operadores entre espacios de funciones. Esto los diferencia de las redes neuronales, que solo aproximan los datos contenidos en una malla, lo que limita su capacidad para resolver este tipo de problemas.

Además, los operadores neuronales tienen la propiedad de invarianza a la resolución, lo que hace que el mismo operador pueda ser entrenado y evaluado en mallas de diferentes tamaños, lo cual es una ventaja frente a las redes clásicas, pues estas deben ser entrenadas y evaluadas en la misma malla debido a su construcción.

Por otro lado, las redes neuronales clásicas solo resuelven una instancia del problema  $Lu = f$ , mientras que los operadores neuronales aprenden una familia paramétrica de EDPs, variando la  $f$ . De esta forma, si nuestro objetivo es aprender una familia o tipo de problema, los operadores neuronales resultan la mejor opción, en cambio, si tan solo queremos resolver una ecuación concreta, nos serán más útiles las redes clásicas.

---

<sup>8</sup>Las PINNs (*Physics-Informed Neural Networks*) incorporan la ecuación diferencial en la función de pérdida. Las Redes Neuronales Convolucionales emplean convoluciones para extraer patrones locales en los datos.

## Operadores Neuronales frente a Métodos Numéricos Clásicos

Los Métodos Numéricos de resolución de EDPs (como el Método de los Elementos Finitos o el de las Diferencias Finitas<sup>9</sup>) transforman una EDP en un problema que pueda resolverse mediante un procedimiento numérico previamente establecido, requiriendo además la forma explícita de la EDP. Por el contrario, los operadores neuronales se entrenan para aproximar directamente el operador de solución a partir de los datos de entrenamiento.

Como las Redes Neuronales Clásicas, los Métodos Numéricos Clásicos solo resuelven una instancia del problema, no una familia de problemas dependientes de un parámetro.

Una desventaja de los operadores neuronales es su estructura de *caja negra*, lo que hace muy complicado saber por qué eligió los parámetros finales en el entrenamiento, en los Métodos Numéricos sabemos perfectamente qué problema estamos tratando.

Además, los Métodos Numéricos son, en especial si la malla es muy fina, computacionalmente más costosos a la hora de ser evaluados. Los operadores neuronales son también muy costosos en el entrenamiento pero muy rápidos en la inferencia.

| Redes neuronales frente a operadores neuronales  |                                                       |
|--------------------------------------------------|-------------------------------------------------------|
| Redes neuronales tradicionales                   | Operadores neuronales                                 |
| Discretización fija                              | Invariancia respecto a la resolución                  |
| Maapeo de vector a vector                        | Maapeo de función a función                           |
| Generalización limitada                          | Aproximación universal                                |
| Entrenamiento dependiente de la resolución       | Entrenamiento e inferencia en diferentes resoluciones |
| Salidas discretas                                | Salidas como funciones continuas                      |
| Resuelven una única instancia del problema       | Familia parametrizada de problemas                    |
| Métodos numéricos frente a operadores neuronales |                                                       |
| Métodos numéricos tradicionales                  | Operadores neuronales                                 |
| Resuelven única una instancia concreta           | Aprenden operadores de solución                       |
| Requieren una forma explícita de la EDP          | Caja negra respecto de sus parámetros                 |
| Lentos en mallas finas                           | Rápidos en distintas resoluciones                     |
| Alto coste computacional                         | Inferencia rápida tras el entrenamiento               |

**Tabla 1.4:** Comparación entre redes neuronales, métodos numéricos y operadores neuronales.

<sup>9</sup>El Método de los Elementos Finitos aproxima la solución mediante una formulación variacional sobre una malla; las Diferencias Finitas sustituyen las derivadas por cocientes discretos.

## Capítulo 2

# Operadores Neuronales de Fourier

Como parte de nuestro estudio de los operadores neuronales, nos centraremos en un tipo muy concreto de ellos, los Operadores Neuronales de Fourier (FNO). A diferencia de los operadores neuronales convencionales, que se entrenan en el dominio  $D \subset \mathbb{R}^n$ , el FNO se entrena en el **dominio de las frecuencias**, pasando a este mediante la transformada de Fourier. Esto permite aprender el comportamiento global del operador solución, reducir la complejidad, pues el operador de Fourier transforma convoluciones en multiplicaciones, y generalizar entre resoluciones, ya que el modelo no depende estrictamente de la malla (invarianza respecto a la discretización).

El FNO, representado en la Figura 2.1, se define como la composición

$$Q \circ \sigma(W_L + K_L) \circ \cdots \circ \sigma(W_1 + K_1) \circ P$$

donde:

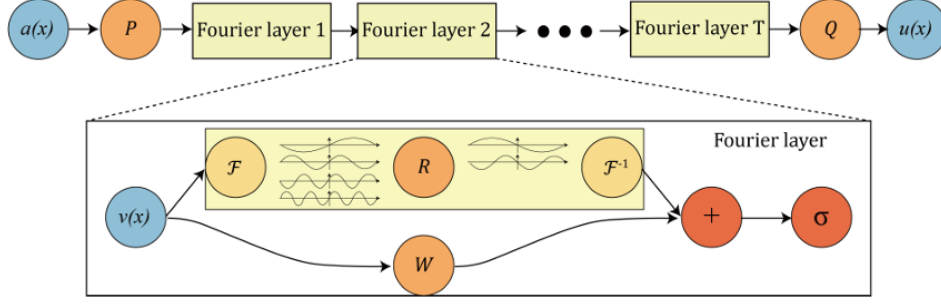
- $K_\ell$  son convoluciones en el dominio de las frecuencias.
- $P$  y  $Q$  son redes neuronales de *lifting* y proyección que codifican la función de menor dimensión en un espacio de mayor dimensión latente y la proyectan de nuevo al espacio original, respectivamente. Estas se implementan como perceptrones multicapa (MLPs).
- $W_\ell$  son transformaciones lineales, implementadas como MLPs .
- $\sigma$  denota una función de activación no lineal.

Los parámetros entrenables del FNO son parámetros de  $P$ ,  $Q$ ,  $W_\ell$  y  $K_\ell$ .

Las **convoluciones espectrales** en las capas de Fourier permiten una mezcla no local de la información a través de las frecuencias [7].

Las transformaciones lineales  $W_\ell$ , de tamaño  $d_{output} \times d_{input}$  la dimensión de los canales, son **conexiones residuales**, ver [arquitectura ResNet](#), que pueden influir y representar variaciones de mayor frecuencia en los datos, complementando así las convoluciones espectrales al enriquecer la representación con detalles de alta frecuencia que pueden perderse en un entrenamiento únicamente espectral.

Finalmente, las capas de **lifting** y **proyección** expanden la entrada a un espacio latente de mayor dimensión para capturar interacciones complejas de características y luego la comprimen de nuevo a la dimensión objetivo.



**Figura 2.1:** Se inicia con una función de entrada  $a$ . Se eleva a un espacio de mayor dimensión mediante un operador puntual  $P$ . Se aplican  $T$  capas de operadores integrales y funciones de activación. Se proyecta de nuevo a la dimensión objetivo mediante una red neuronal  $Q$ . Salida  $u$ . **Capas de Fourier:** comenzando desde la entrada  $v$ . **Arriba:** se aplica la transformada de Fourier  $\mathcal{F}$ ; se utiliza un operador de Fourier  $R$  sobre los modos de menor frecuencia, que también filtra los modos de mayor frecuencia; a continuación se aplica la transformada inversa  $\mathcal{F}^{-1}$ . **Abajo:** se aplica una transformación lineal local  $W$ .

## Convolución en el dominio de las frecuencias

Para simplificar, supondremos que  $D = \mathbb{T}^d$  es el toro unidad y que todas las funciones son complejas. Sea

$$\mathcal{F} : L^2(D; \mathbb{C}^n) \rightarrow \ell^2(\mathbb{Z}^d; \mathbb{C}^n)$$

la transformada de Fourier de una función  $v : D \rightarrow \mathbb{C}^n$  y  $\mathcal{F}^{-1}$  su inversa. Para  $v \in L^2(D; \mathbb{C}^n)$  y  $\hat{v} \in \ell^2(\mathbb{Z}^d; \mathbb{C}^n)$ , tenemos [17]:

$$(\mathcal{F}v)_j(k) = \langle v_j, \varphi_k \rangle_{L^2(D; \mathbb{C})}, \quad j \in \{1, \dots, n\}, \quad k \in \mathbb{Z}^d,$$

$$(\mathcal{F}^{-1}\hat{v})_j(x) = \sum_{k \in \mathbb{Z}^d} \hat{v}_j(k) \varphi_k(x), \quad j \in \{1, \dots, n\}, \quad x \in D,$$

donde, para cada  $k \in \mathbb{Z}^d$ , definimos

$$\varphi_k(x) = e^{2\pi i(k_1 x_1 + \dots + k_d x_d)}, \quad x \in D,$$

**Construcción.** Partimos del operador integral de núcleo  $\mathcal{K}_\theta$ , que actúa sobre una representación latente  $v$ <sup>1</sup> (o vector de características) de la función de entrada agregando información no local a lo largo del dominio espacial:

$$(\mathcal{K}_\theta v)(x) = \int_D \kappa_\theta(x, y) v(y) dy,$$

donde  $\kappa_\theta : D \rightarrow \mathbb{C}^{d_v \times d_v}$  es la función kernel que pretendemos aprender en el entrenamiento. En particular, cuando,  $\kappa_\phi(x, y) = \kappa_\phi(x - y)$ , el operador se convierte en una convolución:

<sup>1</sup>Llamaremos  $v$  a cualquiera de las capas  $v_0 \mapsto v_1 \mapsto \dots \mapsto v_T$  que representan las iteraciones del algoritmo.

$$(\mathcal{K}v)(x) = \int_D \kappa_\phi(x-y) v(y) dy = \kappa_\phi * v.$$

Recordemos que la convolución satisface

$$f * g = \mathcal{F}^{-1}(\mathcal{F}(f) \cdot \mathcal{F}(g)),$$

lo que transforma convoluciones en el dominio espacial en productos en el dominio de frecuencias, podemos reescribir el operador como:

$$(\mathcal{K}_\theta v)(x) = \mathcal{F}^{-1}(\mathcal{F}(\kappa_\theta) \cdot \mathcal{F}(v))(x)$$

Esta identidad nos da una ventaja: en lugar de evaluar directamente el operador integral en el dominio físico, la operación se realiza como un simple producto en el dominio de frecuencias. En lugar de aprender directamente el núcleo  $\kappa_\phi$ , el FNO aprende  $R_\theta$  en el dominio de las frecuencias, capturando información global del operador. La igualdad se expresa como:

$$(\mathcal{K}_\phi v)(x) = \mathcal{F}^{-1}(R_\phi \cdot \mathcal{F}(v))(x),$$

donde  $R_\phi : \mathbb{Z}^d \rightarrow \mathbb{C}^{d_v \times d_v}$  es la transformada de Fourier de la función  $\kappa_\theta : D \rightarrow \mathbb{C}^{d_v \times d_v}$  aprendible que actúa linealmente sobre la representación espectral de  $v$ , parametrizada por  $\theta \in \Theta$ .

Al operar directamente sobre la representación espectral, la capa captura correlaciones no locales y dependencias no lineales que serían complejas a través de coordenadas espaciales (y también temporales cuando el tiempo se trata como una dimensión espacial más).

Para **cada frecuencia**  $k \in \mathbb{Z}^d$ , tenemos  $(\mathcal{F}(v))(k) \in \mathbb{C}^{d_v}$  y  $R_\phi(k) \in \mathbb{C}^{d_v \times d_v}$ . Nótese que, dado que asumimos periodicidad en  $\kappa_\theta$ , admite una expansión en series de Fourier, por lo que podemos trabajar con coeficientes discretos.

Para cada frecuencia  $k$ , el modelo aprende una matriz compleja  $R_\phi(k) \in \mathbb{C}^{d_v \times d_v}$ , donde  $d_v$  es el número de canales. Esta matriz establece cómo la información se mezcla entre diferentes canales para esa frecuencia concreta, permitiendo al modelo capturar cómo interactúan las distintas variables físicas o latentes a lo largo del espacio [17].

En este caso,  $R_\phi(k)$  determina qué modos (o frecuencias) de energía se enfatizan o se atenúan y cómo se redistribuyen las correlaciones entre canales.

El **conjunto de matrices**  $\{R_\phi(k)\}_{k \in \mathbb{Z}^d}$  forma un **tensor** sobre el que actúa el operador en el dominio de Fourier. Sin embargo, el FNO no aprende pesos para todos los modos posibles de  $\mathbb{Z}^d$ . En su lugar, trunca la representación a un subconjunto de los componentes de frecuencia más bajos, es decir, restringimos nuestro tensor  $R_\theta$  al subconjunto del espacio de frecuencias  $Z_{k_{\max}} = \{k \in \mathbb{Z}^d : |k_j| \leq k_{\max,j}, \text{ para cada } j = 1, \dots, d\}$ .

Esta restricción se debe a que en muchos sistemas físicos modelados por EDPs, gran parte de la estructura de la señal se concentra en **frecuencias bajas**, mientras que las altas frecuencias suelen ser ruido o fluctuaciones locales. Aunque estas altas frecuencias pueden recuperarse más tarde mediante las funciones de activación.

Se obtiene esta parametrización de dimensión finita truncando la serie de Fourier de la función de entrada  $v$  a un número máximo de frecuencias  $k_{\max} = |Z_{k_{\max}}|$ . En consecuencia,  $R_\phi$  se parametriza directamente como un tensor complejo de dimensión  $(k_{\max} \times d_v \times d_v)$  que representa una colección de frecuencias truncadas.

**Observación 4.** En el caso real (cuando  $v$  es real), imponemos que  $\kappa_\theta$  sea real forzando simetría:

$$R(-k)_{i,j} = \overline{R(k)_{i,j}}, \quad \forall k \in \mathbb{Z}^d, \quad i, j = 1, \dots, n.$$

## El caso del dominio discreto

Suponiendo que el dominio  $D$  está discretizado en  $n \in \mathbb{N}$  puntos, podemos suponer  $v \in \mathbb{C}^{n \times d_v}$  y  $\mathcal{F}(v) \in \mathbb{C}^{n \times d_v}$ , pues cuando el dominio espacial  $D$  se discretiza en una malla de puntos, la función latente  $v$  se convierte en un array  $v \in \mathbb{R}^{n \times d_v}$ . Dado que convolucionamos  $v$  con una función que solo tiene  $k_{\text{máx}}$  modos de Fourier, truncando los modos de mayor frecuencia podemos obtener  $\mathcal{F}(v) \in \mathbb{C}^{k_{\text{máx}} \times d_v}$ . La multiplicación por el tensor de pesos  $R \in \mathbb{C}^{k_{\text{máx}} \times d_v \times d_v}$  se define como:

$$(R \cdot \mathcal{F}(v))_{k,l} = \sum_{j=1}^{d_v} R_{k,l,j} (\mathcal{F}v)_{k,j}, \quad k = 1, \dots, k_{\text{máx}}, \quad l = 1, \dots, n.$$

La transformada de Fourier  $\mathcal{F}(v)$ , se calcula de manera eficiente a través de la Transformada Rápida de Fourier (FFT), introducida por Cooley y Tukey [6]. En mallas uniformes, cuando la discretización tiene resolución  $r_1 \times \dots \times r_d = n$ , para  $v \in \mathbb{C}^{n \times d_v}$ , con  $k = (k_1, \dots, k_d) \in \mathbb{Z}_{r_1} \times \dots \times \mathbb{Z}_{r_d}$  y  $x = (x_1, \dots, x_d) \in D$ , la FFT  $\hat{\mathcal{F}}$  y su inversa  $\hat{\mathcal{F}}^{-1}$  se definen como:

$$\begin{aligned} (\hat{\mathcal{F}}v)_l(k) &= \sum_{x_1=0}^{r_1-1} \dots \sum_{x_d=0}^{r_d-1} v_l(x_1, \dots, x_d) e^{-2\pi i \sum_{j=1}^d \frac{k_j x_j}{r_j}}, \\ (\hat{\mathcal{F}}^{-1}v)_l(x) &= \sum_{k_1=0}^{r_1-1} \dots \sum_{k_d=0}^{r_d-1} v_l(k_1, \dots, k_d) e^{2\pi i \sum_{j=1}^d \frac{k_j x_j}{r_j}}. \end{aligned}$$

Para  $l = 1, \dots, n$ , el conjunto de frecuencias truncadas viene dado por:

$$Z_{k_{\text{máx}}} = \{(k_1, \dots, k_d) \in \mathbb{Z}_{r_1} \times \dots \times \mathbb{Z}_{r_d} : |k_j| \leq k_{\text{máx},j}, \quad j = 1, \dots, d\}.$$

Mediante esta operación, en cada frecuencia, la matriz compleja  $R_\phi(k)$  mezcla la información entre canales. Esto acopla de manera efectiva las variables físicas. La operación es lineal en el dominio de frecuencias pero no local en el dominio físico.

## Elección de $R_\phi$

En general,  $R_\phi$  puede definirse como una función dependiente de  $\mathcal{F}(a)$ . Podemos definir  $R_\phi : \mathbb{Z}^d \times \mathbb{C}^{d_a} \rightarrow \mathbb{C}^{n \times n}$  como una aplicación que mapea  $(k, (\mathcal{F}a)(k))$  a los valores de los modos de Fourier correspondientes. En la actualidad, se manejan las siguientes parametrizaciones [16]:

- **Directa.** Definir los parámetros  $\phi_k \in \mathbb{C}^{n \times n}$  para cada número de onda  $k$ :

$$R_\phi(k, (\mathcal{F}a)(k)) := \phi_k.$$

- **Lineal.** Definir los parámetros  $\phi_{k,1} \in \mathbb{C}^{n \times n \times d_a}$  y  $\phi_{k,2} \in \mathbb{C}^{n \times n}$  para cada número de onda  $k$ :

$$R_\phi(k, (\mathcal{F}a)(k)) := \phi_{k,1}(\mathcal{F}a)(k) + \phi_{k,2}.$$

- **Red neuronal feed-forward.** Sea  $\Phi_\phi : \mathbb{Z}^d \times \mathbb{C}^{d_a} \rightarrow \mathbb{C}^{n \times n}$  una red neuronal con parámetros  $\phi$ :

$$R_\phi(k, (\mathcal{F}a)(k)) := \Phi_\phi(k, (\mathcal{F}a)(k)).$$

Siendo la más usada la directa y la parametrización mediante red neuronal *feed-forward* la menos usada debido a su rendimiento. Esto probablemente se deba a la estructura discreta del espacio  $\mathbb{Z}^d$ ; puesto que las redes neuronales no se adaptan bien a esta estructura.

## Complejidad computacional del FNO

El tensor de pesos  $R_\phi$  contiene  $k_{\max} < n$  modos, por lo que la multiplicación interna tiene complejidad  $\mathcal{O}(k_{\max})$ . Por tanto, la mayor parte del coste computacional reside en el cálculo de  $\mathcal{F}(v)$  y su inversa. Las transformadas de Fourier generales tienen complejidad  $\mathcal{O}(n^2)$ ; sin embargo, al truncar la serie, la complejidad pasa a ser  $\mathcal{O}(nk_{\max})$ , mientras que la FFT tiene complejidad  $\mathcal{O}(n \log n)$ . Se observa que las FFT son muy eficientes, aunque se requieren de una discretización uniforme. Por tanto puede afirmarse que el FNO tiene una complejidad *cuasi-lineal* [17].

## Invarianza a la resolución

La transformada de Fourier se aplica a las funciones de entrada, lo que hace que el FNO pase a trabajar con los coeficientes de los modos de frecuencia (un número finito de ellos) en lugar de con puntos en mallas discretas en el espacio. Debido a esto, el FNO es independiente de la malla por construcción, lo que provoca que, una vez entrenado, el mismo modelo pueda generalizar a entradas de diferentes resoluciones entrenándose en mallas gruesas y aplicándose directamente a mallas más finas sin necesidad de reentrenamiento, lo que se conoce como *zero-shot super resolution*, manteniendo predicciones consistentes entre distintas discretizaciones, ya que simplemente está evaluando la misma función a mayor resolución.

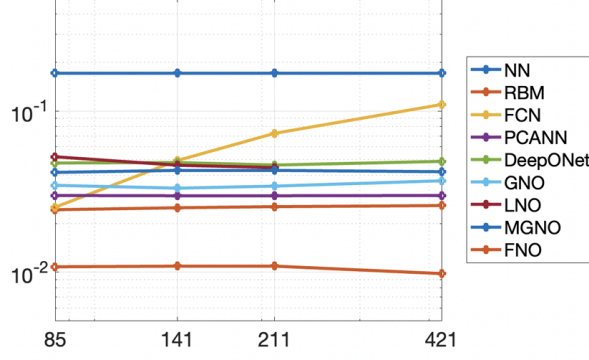
Por el contrario, los métodos basados en redes neuronales con los que se compara presentan un error que crece con la resolución, como se observa en la Figura 2.2.

A parte de esto, la salida puede reconstruirse a cualquier resolución a partir de sus coeficientes en frecuencia mediante la transformada inversa de Fourier (FFT inversa).

Sin embargo, aunque estas pruebas experimentales sean interesantes, investigaciones de 2026 [21] muestran que la realidad es que la superresolución *zero-shot* no está garantizada en general: puede fallar incluso en casos simples y solo puede ser cierta asumiendo más condiciones, como por ejemplo que las salidas del operador cumplan la condición de Hölder, esto es, si existen constantes  $C \geq 0$  y  $\alpha > 0$  de modo que

$$|u(x) - u(y)| \leq C \|x - y\|^\alpha, \quad \forall x, y \in D.$$

Este teorema muestra que la superresolución no se cumple para un conjunto muy simple como son los operadores de rango 1, ver Anexo. Por tanto no es un resultado general.



**Figura 2.2:** Benchmarks en el flujo de Darcy para diferentes resoluciones; entrenamiento y test en la misma resolución. Se ve que el FNO no aumenta su error aunque aumentemos la resolución.

**Teorema 4** (Imposibilidad de la superresolución *zero-shot*). Sea  $\mathcal{X} = \mathcal{Y} = [0, 1)$ , y sea  $\mathcal{G}$  una clase de operadores lineales de rango 1 de  $L^2(\mathcal{X})$  en  $L^2(\mathcal{Y})$  tal que

$$\|G(a)\|_{L^2(\mathcal{Y})} \leq \|a\|_{L^2(\mathcal{X})}, \quad \forall a \in L^2(\mathcal{X})$$

para todo  $G \in \mathcal{G}$ . Sea  $\mu$  una medida de probabilidad sobre  $L^2(\mathcal{X})$  tal que todo  $a \sim \mu$  satisface

$$c \leq |a(x)| \leq 1, \quad \forall x \in \mathcal{X},$$

para alguna constante  $c > 0$ . Entonces existe un operador real  $G \in \mathcal{G}$  tal que, para cualesquiera  $N_1, N_2 \in \mathbb{N}$ , con  $N_2 = mN_1$ , se cumplen las siguientes afirmaciones.

1. **Aprendizaje perfecto en la malla de entrenamiento.** Existe un estimador

$$\hat{F}'_1 : L^2(\mathcal{X}) \rightarrow L^2(\mathcal{Y}),$$

que depende únicamente de una muestra  $(a_1, u_1)$ , con  $a_1 \sim \mu$  y  $u_1 = G(a_1)$  observado solo sobre la malla de entrenamiento  $\mathcal{Y}_{\text{train}} := \{0, \frac{1}{N_1}, \dots, \frac{N_1-1}{N_1}\} \subset \mathcal{Y}$ , tal que

$$\mathbb{E}_{a \sim \mu} \left[ \frac{1}{|\mathcal{Y}_{\text{train}}|} \sum_{y \in \mathcal{Y}_{\text{train}}} \left( \hat{F}'_1(a)(y) - G(a)(y) \right)^2 \right] = 0.$$

2. **Fallo de la superresolución *zero-shot*.** Para todo estimador

$$\hat{F}_n : L^2(\mathcal{X}) \rightarrow L^2(\mathcal{Y})$$

entrenado con muestras

$$\{(a_i, u_i)\}_{i=1}^n,$$

donde  $a_i \sim \mu$  y cada salida  $u_i = G(a_i)$  se observan únicamente sobre la malla de entrenamiento  $\mathcal{Y}_{\text{train}}$ , el error esperado en el test en la malla más fina  $\mathcal{Y}_{\text{test}} := \{0, \frac{1}{N_2}, \dots, \frac{N_2-1}{N_2}\} \subset \mathcal{Y}$  satisface

$$\mathbb{E}_{a \sim \mu} \left[ \frac{1}{|\mathcal{Y}_{\text{test}}|} \sum_{y \in \mathcal{Y}_{\text{test}}} \left( \hat{F}_n(a)(y) - G(a)(y) \right)^2 \right] \geq \frac{c^2}{4} \left( 1 - \frac{1}{m} \right).$$

Esta cota inferior indica que, aunque puede existir un modelo cuyo error promedio sea cero en los datos de entrenamiento, la superresolución en el test sin entrenamiento previo es imposible en este contexto.

## Extensión a geometrías del dominio más generales

El modelo de operador neuronal de Fourier se define sobre operaciones en el espacio de Fourier, acompañadas de operadores residuales locales de tipo ResNet y, potencialmente, términos de sesgo. Estas operaciones están definidas principalmente en geometrías generales, espacios de funciones y discretizaciones arbitrarias. No se limitan a dominios rectangulares, funciones periódicas ni mallas uniformes. Aunque existe una manera de extender el operador a otros dominios:

Dada una variedad compacta  $D = \mathcal{M}$ , podemos embeberla en un cubo periódico (toro):

$$i : \mathcal{M} \rightarrow \mathbb{T}^d,$$

Dada una función  $f \in L^2(\mathcal{M}, \mathbb{R})$ , podemos definir su extensión rellenando con ceros. Definimos esta extensión  $\tilde{f} \in L^2(\mathbb{T}^d, \mathbb{R})$  como:

$$\tilde{f}(x) = \begin{cases} f(x) & \text{si } x \in \mathcal{M} \\ 0 & \text{si } x \in \mathbb{T}^d \setminus \mathcal{M} \end{cases}$$

esta extensión es válida incluso si  $f(x)$  no se anula en la frontera  $\partial\mathcal{M}$ , puesto que la discontinuidad introducida no afecta la integrabilidad de la función. Así hemos convertido una función definida sobre un dominio compacto cualquiera en una periódica a la que podemos aplicar toda la teoría estudiada anteriormente.

La optimización de la función pérdida se calcula únicamente sobre el dominio original  $\mathcal{M}$ . El operador neuronal de Fourier generará automáticamente una extensión suave al dominio ampliado a  $\mathbb{T}^d$  en el espacio de salida.

# Capítulo 3

## Problemas Clásicos en Física Matemática

Reformularemos algunos problemas conocidos en Física para poder resolverlos aproximando su operador solución con un operador neuronal. Esto nos permitirá evidenciar el potencial que tiene este nuevo paradigma a la hora de predecir fenómenos físicos complejos.

### Ecuación de Poisson

Primero consideramos la ecuación de Poisson unidimensional con condición de contorno nula:

$$\begin{aligned} -\frac{d^2}{dx^2}u(x) &= f(x), \quad x \in (0, 1) \\ u(0) &= u(1) = 0 \end{aligned}$$

para alguna función fuente  $f : (0, 1) \rightarrow \mathbb{R}$ . En particular, para  $D(L) := H_0^1((0, 1); \mathbb{R}) \cap H^2((0, 1); \mathbb{R})$ , se tiene  $L : D(L) \rightarrow L^2((0, 1); \mathbb{R})$  definida como  $-d^2/dx^2$ , nótese que esto no depende de ningún parámetro  $a \in \mathcal{A}$  en este caso [16].

Si  $f \in L^2((0, 1); \mathbb{R})$ , el operador solución  $\mathcal{G}^\dagger : L^2((0, 1); \mathbb{R}) \rightarrow H_0^1((0, 1); \mathbb{R})$  dado por  $\mathcal{G}^\dagger : f \mapsto u$ , es el que debemos aprender en la topología inducida por la norma de Bochner.

La medida de probabilidad normalmente utilizada en este caso es  $\mu = \mathcal{N}(0, C)$  sobre  $\mathcal{A}$ , donde  $C = (L + I)^{-2}$ , definida mediante la teoría espectral de operadores autoadjuntos<sup>1</sup>.

---

<sup>1</sup>La expresión  $C = (L + I)^{-2}$  debe entenderse como un operador construido a partir de dos operadores autoadjuntos,  $C$  y  $I$ , es decir, si  $L(\varphi_k) = \lambda_k \varphi_k$ , entonces  $C(\varphi_k) = (\lambda_k + 1)^{-2} \varphi_k$ , siendo  $\varphi_k$  los autovectores y  $\lambda_k$  los respectivos autovalores del operador  $L$ . De este modo, para generar  $f$ , se escribe como serie aleatoria:

$$f(x) = \sum_{k=1}^{\infty} \alpha_k \varphi_k(x), \quad \text{donde} \quad \alpha_k \sim \mathcal{N}(0, (\lambda_k + 1)^{-2})$$

como no se pueden sumar infinitos términos, se trunca la serie:

$$f_M(x) = \sum_{k=1}^M \alpha_k \varphi_k(x)$$

Como la varianza de  $\alpha_k$  es  $(\lambda_k + 1)^{-2}$ , su desviación típica es  $(\lambda_k + 1)^{-1}$ . Por tanto, se generan

En este contexto, el operador solución  $\mathcal{G}^\dagger$  viene dado por

$$\mathcal{G}^\dagger(f) = \int_0^1 G(\cdot, y) f(y) dy$$

donde

$$G(x, y) = \frac{1}{2} (x + y - |y - x|) - xy, \quad \forall (x, y) \in [0, 1]^2$$

es la función de Green. Nótese que aunque  $\mathcal{G}^\dagger$  es un operador lineal, aunque la función de Green  $G$  no sea lineal como función de sus argumentos. Vamos ahora a construir su operador neuronal correspondiente.

Consideraremos una única capa con  $\sigma_1 = \text{Id}$ ,  $P = \text{Id}$ ,  $Q = \text{Id}$ ,  $W_0 = 0$ ,  $b_0 = 0$ , y

$$\mathcal{K}_\theta(f) = \int_0^1 \kappa_\theta(\cdot, y) f(y) dy$$

donde  $\kappa_\theta : \mathbb{R}^2 \rightarrow \mathbb{R}$  será parametrizada como una red neuronal estándar con parámetros  $\theta$ .

Para construir el conjunto de datos de entrenamiento, las soluciones de la ecuación se pueden obtener evaluando la función de Green (que es conocida analíticamente) en una malla uniforme. Como puede verse en 3.1, el operador neuronal provee una aproximación casi perfecta en la norma de Bochner de la función de Green.

Para comprobar si el operador verdadero aproxima bien también en la topología uniforme, comprobamos si el núcleo lineal  $\kappa_\theta$  aproxima la función de Green para este problema. Para ver por qué esto es suficiente, sea  $K \subset L^2([0, 1]; \mathbb{R})$  un conjunto compacto, es decir,

$$\|f\|_{L^2([0,1];\mathbb{R})} \leq M, \quad \forall f \in K$$

y supongamos que

$$\sup_{(x,y) \in [0,1]^2} |\kappa_\theta(x, y) - G(x, y)| < \frac{\varepsilon}{M}$$

para algún  $\varepsilon > 0$ . Entonces es fácil ver que

$$\sup_{f \in K} \|\mathcal{G}^\dagger(f) - \mathcal{K}_\theta(f)\|_{L^2([0,1];\mathbb{R})} < \varepsilon.$$

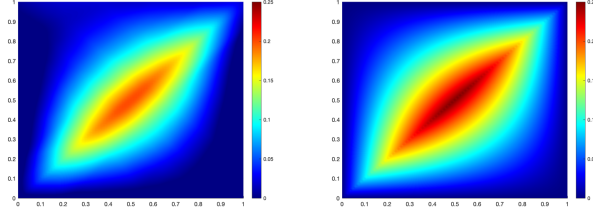
En particular, obtenemos una aproximación en la topología de convergencia uniforme sobre conjuntos compactos, mientras que habíamos entrenado únicamente en la topología

---

variables aleatorias normales estándar  $\xi_k \sim \mathcal{N}(0, 1)$ , y se define  $\alpha_k = (\lambda_k + 1)^{-1} \xi_k$ . De este modo, la función aleatoria truncada se construye como

$$f_M(x) = \sum_{k=1}^M \frac{\xi_k}{\lambda_k + 1} \sqrt{2} \sin(k\pi x).$$

Esta es la forma práctica de generar una función aleatoria  $f$ , según la distribución  $f \sim \mathcal{N}(0, (L + I)^{-2})$ .



**Figura 3.1:** Núcleo para la función de Green unidimensional. **Izquierda:** función núcleo aprendida; **Derecha:** función de Green analítica.

de la norma de Bochner. La Figura 3.1 muestra que  $\kappa_\theta$  efectivamente aproxima la función de Green.

### Ecuación de Burgers

Consideramos la ecuación de Burgers viscosa unidimensional:

$$\begin{aligned} \frac{\partial}{\partial t} u(x, t) + \frac{1}{2} \frac{\partial}{\partial x} (u(x, t))^2 &= \nu \frac{\partial^2}{\partial x^2} u(x, t), \quad x \in (0, 2\pi), t \in (0, \infty) \\ u(x, 0) &= u_0(x), \quad x \in (0, 2\pi) \end{aligned}$$

con condiciones de contorno periódicas y una viscosidad fija  $\nu$ . El mapa de flujo asociado se define como  $\Psi : L^2((0, 2\pi); \mathbb{R}) \times \mathbb{R}_+ \rightarrow H^s((0, 2\pi); \mathbb{R})$ , para cualquier  $s > 0$

$$\Psi(u_0, t) = u(\cdot, t), \quad t > 0.$$

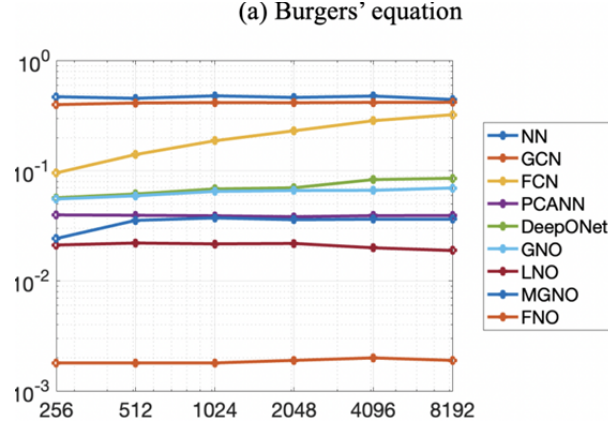
Consideramos el operador de solución definido al evaluar  $\Psi$  en un tiempo fijo. Fijamos cualquier  $s \geq 0$ . Entonces podemos definir  $\mathcal{G}^\dagger : L^2((0, 2\pi)) \rightarrow H^s((0, 2\pi); \mathbb{R})$  mediante

$$\mathcal{G}^\dagger : u_0 \mapsto \Psi(u_0, 1).$$

Tomando una medida de probabilidad cualquiera  $\mu = \mathcal{N}(0, C)$  sobre  $L^2((0, 2\pi); \mathbb{R})$ , elegimos la condición inicial para la ecuación mediante  $u_0 \sim \mu$ , como en el caso anterior.

Para crear el conjunto de datos utilizado para el entrenamiento, generamos las soluciones obedeciendo la distribución  $u_0 \sim \mu$ , y seguidamente para cada  $u_0$  generada resolvemos la ecuación mediante métodos clásicos para obtener  $\Psi(u_0, 1) = u(x, 1)$ , generando los pares de entrenamiento  $(u_0, u(x, 1))$ .

En la Figura 3.2 extraída de [16], podemos ver una comparativa entre distintos tipos de operadores neuronales a la hora de resolver la ecuación de Burgers con otros modelos basados en redes neuronales conforme va aumentando la resolución. En ella se representa la diferencia entre la solución real para una entrada y la predicción obtenida por el modelo. Observamos nuevamente la invarianza del error con respecto a la resolución.



**Figura 3.2:** Error en la ecuación de Burgers para diferentes resoluciones; entrenamiento y test en la misma resolución.

### Ecuación de Navier–Stokes

Consideramos la ecuación de Navier–Stokes bidimensional para un fluido viscoso e incompresible [8]:

$$\frac{\partial u}{\partial t}(x, t) + u(x, t) \cdot \nabla u(x, t) = -\nabla p(x, t) + \nu \nabla^2 u(x, t) + f(x), \quad x \in \mathbb{T}^2, t \in (0, \infty)$$

$$\nabla \cdot u(x, t) = 0, \quad x \in \mathbb{T}^2, t \in (0, \infty)$$

$$u(x, 0) = u_0(x), \quad x \in \mathbb{T}^2$$

donde  $\mathbb{T}^2$  es el toro unidad, es decir,  $[0, 1]^2$  con condiciones de contorno periódicas, y  $\nu \in \mathbb{R}$  es una viscosidad fija. En esta ecuación  $u : \mathbb{T}^2 \times \mathbb{R}_+ \rightarrow \mathbb{R}^2$  es el campo de velocidades,  $p : \mathbb{T}^2 \times \mathbb{R}_+ \rightarrow \mathbb{R}$  es el campo de presiones, y  $f : \mathbb{T}^2 \rightarrow \mathbb{R}$  es una función de fuerza externa.

La primera ecuación es conocida como *ecuación de momento* y se obtiene de la aplicación de la segunda ley de Newton<sup>2</sup> aplicada a un elemento de fluido. Notemos que  $u \cdot \nabla u$  es un término no lineal, lo cual hace que la resolución analítica de la ecuación sea compleja y, en muchos casos, imposible de obtener.

La segunda ecuación se denomina *de continuidad* y se basa en el principio de conservación de la masa<sup>3</sup> aplicada a una fluido en movimiento.

<sup>2</sup>La ecuación anterior se obtiene aplicando la segunda ley de Newton,  $m\mathbf{a} = \sum \mathbf{F}$ , a un elemento de fluido. Como  $\mathbf{u}(\mathbf{x}, t)$  es el campo de velocidades, la aceleración de una partícula fluida viene dada por la derivada:

$$\mathbf{a} = \frac{D\mathbf{u}}{Dt} = \frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u}.$$

Este término se multiplica por la densidad  $\rho$ , ya que se trabaja con la ecuación de movimiento por unidad de volumen. En el segundo miembro se agrupan las fuerzas que actúan sobre el fluido: la fuerza debida al gradiente de presión, la fuerza viscosa y una fuerza externa, habitualmente asociada a la gravedad. De este modo se obtiene

$$\rho \left( \frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = -\nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f}.$$

<sup>3</sup>La ecuación original tiene la forma  $\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0$ , aplicando la regla del producto  $\frac{\partial \rho}{\partial t} + \mathbf{u} \cdot \nabla \rho + \rho \nabla \cdot \mathbf{u} = 0$ , pero como el fluido es incompresible su densidad es constante, anulándose los dos primeros términos.

Para simplificar el problema, vamos a reescribir esta ecuación en términos de la vorticidad y función de corriente asociada.

Sea  $\nabla \times u : \mathbb{T}^2 \times \mathbb{R}_+ \rightarrow \mathbb{R}^3$  el campo de vorticidad. Nótese que, cuando se visualiza en tres dimensiones,  $u = (u_1(x_1, x_2), u_2(x_1, x_2), 0)$ , se tiene en nuestro caso  $\nabla \times u = (0, 0, w)$ . Tomamos  $w$  como la componente fuera del plano del campo de vorticidad.

La función de corriente  $\psi$  está relacionada con la velocidad por  $u = \nabla^\perp \psi$ , imponiendo la condición de incompresibilidad. También consideraremos el rotacional de  $f$ , de forma que  $\nabla \times f = (0, 0, g)$ . Estas relaciones permiten reformular la ecuación de esta manera:

$$\begin{aligned} \frac{\partial w}{\partial t}(x, t) + \nabla^\perp \psi \cdot \nabla w(x, t) &= \nu \Delta w(x, t) + g(x), \quad x \in \mathbb{T}^2, t \in (0, \infty) \\ -\Delta \psi &= w, \quad x \in \mathbb{T}^2, t \in (0, \infty) \\ w(x, 0) &= w_0(x), \quad x \in \mathbb{T}^2 \end{aligned}$$

Siendo  $g(x)$  el término de fuerza externa.

Sea  $\Psi : L^2(\mathbb{T}^2; \mathbb{R}) \times \mathbb{R}_+ \rightarrow H^s(\mathbb{T}^2; \mathbb{R})$ , para cualquier  $s > 0$ , el mapa de flujo asociado, en particular,

$$\Psi(w_0, t) = w(\cdot, t), \quad t > 0.$$

Obsérvese que esto está bien definido para cualquier  $w_0 \in L^2(\mathbb{T}^2; \mathbb{R})$ .

Definimos entonces dos nociones del operador solución. En la primera, procedemos imponiendo como variable la condición inicial para algún  $T > 0$  fijo

$$\mathcal{G}^\dagger : L^2(\mathbb{T}^2; \mathbb{R}) \rightarrow H^s(\mathbb{T}^2; \mathbb{R}) \quad \text{dado por} \quad \mathcal{G}^\dagger : w_0 \mapsto \Psi(w_0, T)$$

En la segunda, haremos un mapeo de una parte inicial de la trayectoria a una parte posterior de la misma. En particular, definimos para algún  $T > 10$

$$\mathcal{G}^\dagger : L^2(\mathbb{T}^2; \mathbb{R}) \times C((0, 10]; H^s(\mathbb{T}^2; \mathbb{R})) \rightarrow C((10, T]; H^s(\mathbb{T}^2; \mathbb{R}))$$

$$\mathcal{G}^\dagger : \left( w_0, \Psi(w_0, t)|_{t \in (0, 10]} \right) \mapsto \Psi(w_0, t)|_{t \in (10, T]}$$

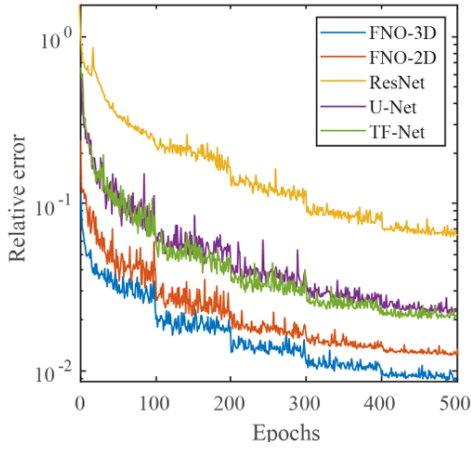
Tomando una medida de probabilidad cualquiera  $\mu = \mathcal{N}(0, C)$  sobre  $L^2(\mathbb{T}^2; \mathbb{R})$ , para crear el conjunto de datos utilizado en el entrenamiento, obteniendo  $w_0 \sim \mu$  como en los casos anteriores. Terminamos de definir la medida sobre el espacio de entrada mediante la imagen  $\Psi(w_0, t)|_{t \in (0, 10]}$ , obtenida a partir de simuladores numéricos clásicos imponiendo  $w_0$  como condición inicial.

En la Figura 3.3, extraída de [16] se puede ver una comparativa con otros métodos de resolución de la ecuación de Navier–Stokes. En ella se muestra el error cuadrático medio de los modelos obtenido en cada época de entrenamiento. Merece la pena mencionar dos formas distintas de aplicar el FNO: el FNO-2D y el FNO-3D.

Los modelos FNO-2D y FNO-3D se diferencian principalmente en el tratamiento del tiempo. En el 2D, la red aplica convoluciones sobre las coordenadas espaciales y predice

la evolución de forma autorregresiva, es decir, a partir de varios estados temporales previos calcula el siguiente, repitiendo el proceso para construir la trayectoria completa, es decir, utiliza cada predicción obtenida como entrada para calcular el siguiente instante temporal.

En cambio, en el 3D considera el tiempo como una dimensión adicional y aplica convoluciones directamente sobre las coordenadas espacio-temporales, prediciendo de una vez una trayectoria futura completa. Por ello los métodos 2D funcionan mejor para secuencias temporales cortas, mientras que el método 3D es más fácil de entrenar en secuencias largas. En este caso, Figura 3.3, para el 2D se fijó en  $k_{\text{máx},1} = 12$ ,  $d_u = 32$  y para el 3D se fijó en  $k_{\text{máx},1} = 12$ ,  $d_u = 20$ .



**Figura 3.3:** Curvas de aprendizaje en Navier–Stokes con  $\nu = 10^{-3}$  para diferentes métodos. Entrenamiento y test en la misma resolución.

# Capítulo 4

## Programación de un Operador Neuronal

En este capítulo llevaremos a la práctica los conceptos desarrollados anteriormente. Para ello, entrenaremos un operador neuronal y evaluaremos su capacidad de aproximación mediante métricas de entrenamiento y prueba.

En este caso, se ha elegido trabajar con el Operador Neuronal de Fourier aplicado a las ecuaciones de Navier–Stokes en dos dimensiones, que modelan la evolución de un fluido incompresible, expresando la vorticidad en cada punto.

Para la implementación del código se han tomado como referencia las librerías de Python Pytorch y NeuralOperator [14][1], que ofrecen herramientas para definir, entrenar y evaluar arquitecturas de Operadores Neuronales.

### Carga y preparación de la base de datos

En primer lugar, cargaremos el dataset de entrenamiento de las ecuaciones de Navier-Stokes en dos dimensiones para un fluido incompresible. Esta base de datos consta de pares de entrada y salida. La entrada y la salida son ambos campos bidimensionales definidos sobre una malla de puntos finita [15], con un solo canal de datos que describe la vorticidad, es decir, un escalar  $w$ , pues recordemos que el campo de vorticidad era  $\nabla \times u = (0, 0, w)$ . Matemáticamente representan campos escalares definidos sobre el toro  $\mathbb{T}^2$ .

Cargamos 1000 muestras para entrenamiento y 50 para test de  $128 \times 128$  píxeles, o dicho de otra forma, puntos de la malla.

[1]:

```
from neuralop.data.datasets import NavierStokesDataset

dataset = NavierStokesDataset(root_dir="/content/data",
    ↪train_resolution=128, n_train=1000, batch_size=32, n_tests=[50],
    ↪test_batch_sizes=[32], test_resolutions=[128])
```

Loading test db for resolution 128 with 50 samples

Como no podemos evaluar todos los ejemplos de entrenamiento a la vez, dividimos cada época en lotes (*batches*) de 32 imágenes.

```
[2]: # Preparamos los datos de entrenamiento y test para que sean
      ↪ entregados en pequeños lotes (batches)
import torch
from torch.utils.data import DataLoader

# Creamos el cargador de entrenamiento mezclando los datos con
      ↪ shuffle=True
train_loader = DataLoader(dataset.train_db, batch_size=32, shuffle=True)

# Creamos el cargador de test. Como 'test_dbs' es un diccionario, #
      ↪ le pedimos la base de datos de la resolución 128 que descargamos.
test_loader_128 = DataLoader(dataset.
      ↪ test_dbs[128], batch_size=32, shuffle=False)

# El Trainer necesita un diccionario de cargadores indexado por su
      ↪ resolución, en este caso solo hay una, la de 128:
test_loaders = {128: test_loader_128}
```

Establecemos la configuración para que el entrenamiento ejecute en una GPU y comprobamos la forma del dataset.

```
[3]: import torch
from torch.utils.data import DataLoader

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

batch = next(iter(train_loader))

print("Forma de x:", batch["x"].shape)
print("Forma de y:", batch["y"].shape)

in_channels = batch["x"].shape[1] # [1] se refiere al número de
      ↪ canales siguiendo el orden [Batch, Canales, Altura, Anchura]
out_channels = batch["y"].shape[1]

print("in_channels:", in_channels)
print("out_channels:", out_channels)
```

Forma de x: torch.Size([32, 1, 128, 128])

Forma de y: torch.Size([32, 1, 128, 128])

in\_channels: 1 (es la dimensión del canal de vorticidad, escalar)

out\_channels: 1 (lo mismo, pues precice la vorticidad en el instante t+1)

Como puede observarse, cada entrada o salida con la que vamos a entrenar al modelo es un tensor de tamaño [32, 1, 128, 128], es decir, 32 ejemplos por batch, y, en cada uno, 1 canal para la vorticidad (escalar), y una matriz 128×128 (la malla).

```
[4]: #Instanciamos el modelo FNO 2d
from neuralop.models import FNO
```

```

model = FNO(n_modes=(16, 16), hidden_channels=32,
            in_channels=in_channels, out_channels=out_channels)

model = model.to(device)

```

Veamos como es la arquitectura del modelo:

```
[5]: print(model)
```

```

FNO(
  (positional_embedding): GridEmbeddingND()
  (fno_blocks): FNOBlocks(
    (convs): ModuleList(
      (0-3): 4 x SpectralConv(
        (weight): DenseTensor(shape=torch.Size([32, 32, 16, 9]),
↪rank=None) ) )
    (fno_skips): ModuleList(
      (0-3): 4 x Flattened1dConv(
        (conv): Conv1d(32, 32, kernel_size=(1,), stride=(1,),
↪bias=False) ) )
    (channel_mlp): ModuleList(
      (0-3): 4 x ChannelMLP(
        (fcs): ModuleList(
          (0): Conv1d(32, 16, kernel_size=(1,), stride=(1,))
          (1): Conv1d(16, 32, kernel_size=(1,), stride=(1,)) ) )
    (channel_mlp_skips): ModuleList(
      (0-3): 4 x SoftGating() ) )
  (lifting): ChannelMLP(
    (fcs): ModuleList(
      (0): Conv1d(3, 64, kernel_size=(1,), stride=(1,))
      (1): Conv1d(64, 32, kernel_size=(1,), stride=(1,)) ) )
  (projection): ChannelMLP(
    (fcs): ModuleList(
      (0): Conv1d(32, 64, kernel_size=(1,), stride=(1,))
      (1): Conv1d(64, 1, kernel_size=(1,), stride=(1,)) ) )

```

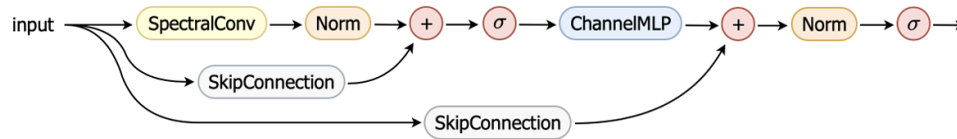
La entrada efectiva tiene tres canales: el valor de la variable de vorticidad y dos coordenadas espaciales. En primer lugar, la red aplica una capa de *lifting*, implementada mediante un **ChannelMLP**, que transforma los 3 canales iniciales en 32 canales latentes.

Hay cuatro bloques de FNO. Cada bloque contiene una **convolución espectral**  $\text{SpectralConv}(v) = \mathcal{F}^{-1}(R_\theta \cdot \mathcal{F}(v))$ , con el tensor  $R_\theta$  representado en **DenseTensor**, que actúa sobre un número finito de frecuencias, en este caso  $16 \times 9$ . En cada una de estas capas, los pesos aprendibles tienen dimensión  $32 \times 32 \times 16 \times 9$ , indicando que el operador transforma 32 canales de entrada en 32 canales de salida para cada frecuencia.

Además de la convolución espectral, cada bloque incluye una **conexión residual**  $Wv$ , llamada **fno-skips**, mediante una convolución  $1 \times 1$ . Seguidamente, se pasa por otra red neuronal llamada **channel-mlp**, junto con otra conexión residual **channel-mlp-skips**,

que reduce primero de 32 a 16 canales y amplía de nuevo a 32. esto permite que los **canales se combinen entre si** antes de pasar el siguiente bloque.

Finalmente, usando otra red neuronal, la capa de *projection* primero amplía de 32 a 64 canales y después reduce a un único canal de salida. Transforma la representación latente de 32 canales en la salida deseada. Por tanto, el modelo aprende un operador que toma una función de entrada definida sobre una malla bidimensional y devuelve una función escalar sobre esa misma malla [7].



```
[6]: # Definimos las pérdidas y métricas

from neuralop.losses import LpLoss

loss_fn = LpLoss(d=2, p=2)

def relative_l2_error(pred, target, eps=1e-12):

    # Error relativo L2 medio por batch.

    pred_flat = pred.reshape(pred.shape[0], -1)
    target_flat = target.reshape(target.shape[0], -1)

    numerator = torch.norm(pred_flat - target_flat, p=2, dim=1)
    denominator = torch.norm(target_flat, p=2, dim=1)

    return torch.mean(numerator / (denominator + eps))

def compute_metrics(pred, target):

    # Calculamos varias métricas útiles para analizar el modelo.

    mse = torch.mean((pred - target) ** 2)
    rel_l2 = relative_l2_error(pred, target)

    return {"mse": mse.item(), "rel_l2": rel_l2.item()}
```

## Entrenamiento del modelo

```
[7]: #Optimizador y scheduler (reduce poco a poco la tasa de aprendizaje)

from torch.optim import AdamW
from torch.optim.lr_scheduler import CosineAnnealingLR
```

```

n_epochs = 100 # Número de épocas
# Usa el gradiente actual y la media de gradientes anteriores
optimizer = AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = CosineAnnealingLR(optimizer, T_max=n_epochs)

```

```

[8]: import os
import time
import torch
import pandas as pd
import matplotlib.pyplot as plt
import imageio.v2 as imageio
# Inicializa como infinito para que cualquier resultado sea mejor
history = []
best_eval_l2 = float("inf")
best_model_path = "fno_navier_stokes_modelo.pt"

for epoch in range(n_epochs): # Por cada época
    start_time = time.time()

    # ENTRENAMIENTO

    model.train() # Activamos el modo entrenamiento del modelo

    train_loss_total = 0.0
    train_mse_total = 0.0
    train_l2_total = 0.0
    train_samples = 0

    for batch in train_loader: # Por cada batch
        x = batch["x"].float().to(device) # Entradas
        y = batch["y"].float().to(device) # Salidas reales

        optimizer.zero_grad() # Borramos los gradientes anteriores

        pred = model(x) # Hacemos una predicción

        loss = loss_fn(pred, y) # Calculamos función de pérdida L2
        loss.backward() # Calculamos los gradientes
        optimizer.step() # Aplicamos el Descenso de gradiente

        batch_size = x.shape[0] # N° de muestras del lote
        train_samples += batch_size

        metrics = compute_metrics(pred.detach(), y)

    train_loss_total += loss.item() * batch_size
    train_mse_total += metrics["mse"] * batch_size

```

```

    train_l2_total += metrics["rel_l2"] * batch_size

scheduler.step() # Reducimos el tamaño del paso

train_loss_avg = train_loss_total / train_samples
train_mse_avg = train_mse_total / train_samples
train_l2_avg = train_l2_total / train_samples

# EVALUACIÓN

model.eval() # Activamos el modo evaluación del modelo

eval_loss_total = 0.0
eval_mse_total = 0.0
eval_l2_total = 0.0
eval_samples = 0

with torch.no_grad():
    for batch in test_loader_128:
        x = batch["x"].float().to(device)
        y = batch["y"].float().to(device)

        pred = model(x) # Hacemos una predicción

        val_loss = loss_fn(pred, y) # Calculamos el error

        batch_size = x.shape[0]
        eval_samples += batch_size

        metrics = compute_metrics(pred, y)
        # Error medio en cada batch
        eval_loss_total += val_loss.item() * batch_size
        eval_mse_total += metrics["mse"] * batch_size
        eval_l2_total += metrics["rel_l2"] * batch_size

# Error medio de toda la época
eval_loss_avg = eval_loss_total / eval_samples
eval_mse_avg = eval_mse_total / eval_samples
eval_l2_avg = eval_l2_total / eval_samples

epoch_time = time.time() - start_time
current_lr = optimizer.param_groups[0]["lr"]

# GUARDAR MÉTRICAS

```

```

    row = {"epoch": epoch, "time_sec": epoch_time, "lr": ↵
↵current_lr, "train_loss_lp": train_loss_avg, "train_rel_l2": ↵
↵train_l2_avg, "train_mse": train_mse_avg, "eval_loss_lp": ↵
↵eval_loss_avg, "eval_rel_l2": eval_l2_avg, "eval_mse": eval_mse_avg,}

    history.append(row)

    # Guardar CSV en cada época
    history_df = pd.DataFrame(history)
    history_df.to_csv("training_history.csv", index=False)

    # Guardar mejor modelo según error relativo L2 en test
    if eval_l2_avg < best_eval_l2:
        best_eval_l2 = eval_l2_avg
        torch.save(model.state_dict(), best_model_path)

    print(f"[{epoch}] "f"time={epoch_time:.2f}s | ↵
↵"f"train_l2={train_l2_avg:.4f} | "f"eval_l2={eval_l2_avg:.4f} | ↵
↵"f"train_mse={train_mse_avg:.6f} | "f"eval_mse={eval_mse_avg:.6f}")

print(f"Mejor modelo guardado en: {best_model_path}")
print("Historial guardado en: training_history.csv")

```

```

[0] time=7.43s | train_l2=0.9513 | eval_l2=0.6146 | train_mse=0.451868 | ↵
↵eval_mse=0.191769
[1] time=5.30s | train_l2=0.5493 | eval_l2=0.5140 | train_mse=0.151545 | ↵
↵eval_mse=0.133581
...
[98] time=5.34s | train_l2=0.0567 | eval_l2=0.0707 | train_mse=0.001753 ↵
↵| eval_mse=0.003089
[99] time=5.34s | train_l2=0.0567 | eval_l2=0.0707 | train_mse=0.001753 ↵
↵| eval_mse=0.003089

```

Mejor modelo guardado en: fno\_navier\_stokes\_modelo.pt

Historial guardado en: training\_history.csv

Observamos que tanto el error de entrenamiento como el de test son bajos, eso significa que no ha habido sobreajuste a los datos de entrenamiento, pues si lo hubiera el error de test sería alto, ya que el modelo no ha visto nunca ese ejemplo.

## Evolución del Error durante el Entrenamiento

Procederemos a graficar el error que hemos obtenido en cada época de entrenamiento.

```

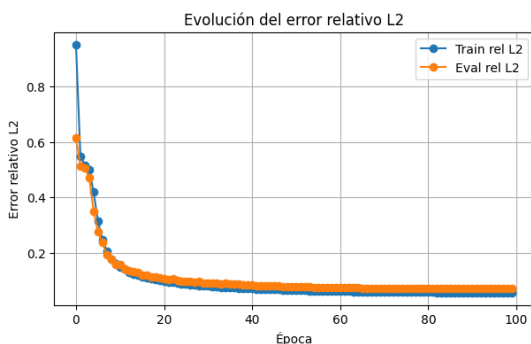
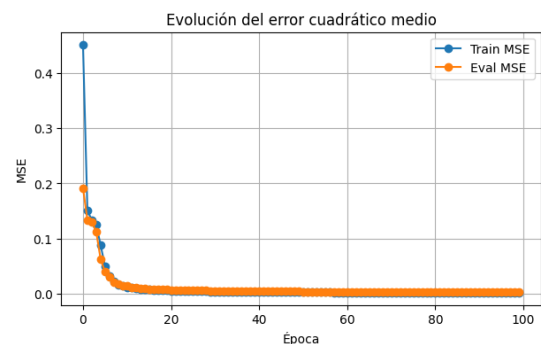
[9]: #Graficar métricas
history_df = pd.read_csv("training_history.csv")

plt.figure(figsize=(7, 4))
plt.plot(history_df["epoch"], history_df["train_rel_l2"], marker="o", ↵
↵label="Train rel L2")

```

```
plt.plot(history_df["epoch"], history_df["eval_rel_l2"], marker="o",
        label="Eval rel L2")
plt.xlabel("Época")
plt.ylabel("Error relativo L2")
plt.title("Evolución del error relativo L2")
plt.legend()
plt.grid(True)
plt.show()
```

```
[10]: plt.figure(figsize=(7, 4))
plt.plot(history_df["epoch"], history_df["train_mse"], marker="o",
        label="Train MSE")
plt.plot(history_df["epoch"], history_df["eval_mse"], marker="o",
        label="Eval MSE")
plt.xlabel("Época")
plt.ylabel("MSE")
plt.title("Evolución del error cuadrático medio")
plt.legend()
plt.grid(True)
plt.show()
```

(a) Evolución del error relativo  $L^2$ 

(b) Evolución del error cuadrático medio

**Figura 4.1:** Métricas de evolución del error del modelo durante el entrenamiento

## Evaluación de Predicciones

```
[11]: # Hacemos una predicción con el modelo entrenado
import torch
import matplotlib.pyplot as plt

indice = 0 # imagen 0 de nuestro dataset de test de 50 imágenes

sample = dataset.test_dbs[128][indice] # Extraemos una muestra del
    dataset

x = sample["x"].unsqueeze(0).float().to(device) # Entrada
y = sample["y"].unsqueeze(0).float().to(device) # Salida real
```

```

model.eval()

with torch.no_grad():
    pred = model(x) # Hacemos una predicción con la entrada

# Extraemos de cada predicción el primer batch y el primer y único
# canal del tensor , por eso escribimos [0,0], siguiendo la convención
# [Batch, Canales, Altura, Anchura]
# esto, por dimensiones, nos da una matriz (una imagen) de 128x128, que
# es nuestra predicción.
x_img = x[0, 0].detach().cpu()
y_img = y[0, 0].detach().cpu()
pred_img = pred[0, 0].detach().cpu()
err_img = torch.abs(pred_img - y_img)

rel_l2 = torch.norm(pred_img - y_img) / torch.norm(y_img)
print(f"Error relativo L2 de la muestra {indice}: {rel_l2.item():.4f}")

plt.figure(figsize=(15, 4))

plt.subplot(1, 4, 1)
plt.imshow(x_img, cmap="viridis")
plt.title("Entrada")
plt.colorbar()
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(y_img, cmap="viridis")
plt.title("Real")
plt.colorbar()
plt.axis("off")

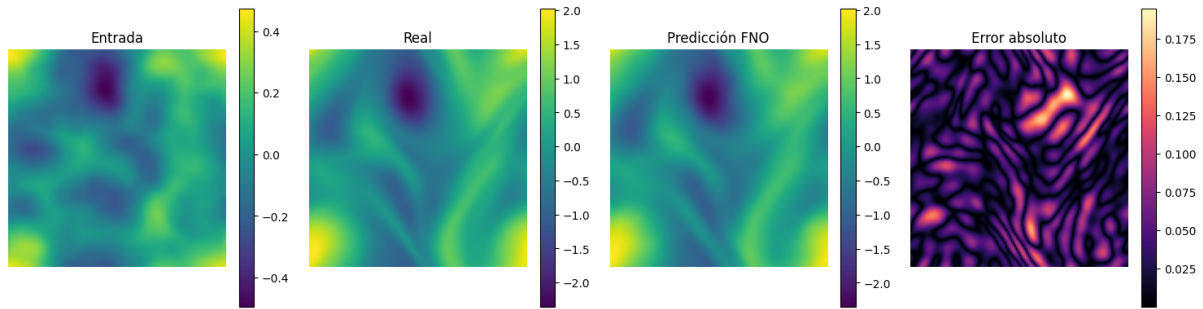
plt.subplot(1, 4, 3)
plt.imshow(pred_img, cmap="viridis")
plt.title("Predicción FNO")
plt.colorbar()
plt.axis("off")

plt.subplot(1, 4, 4)
plt.imshow(err_img, cmap="magma")
plt.title("Error absoluto")
plt.colorbar()
plt.axis("off")

plt.tight_layout()
plt.show()

```

Error relativo L2 de la muestra 0: 0.0660



**Figura 4.2:** Tomando la imagen de la izquierda como entrada, se muestra la predicción de la vorticidad del FNO junto con el valor real observado y el error absoluto entre ambos. Como podemos observar, los Operadores Neuronales tienen una gran capacidad de aproximación.

# Capítulo 5

## Conclusiones

El objetivo principal de este trabajo ha sido dar una idea general de los operadores neuronales, desde sus fundamentos matemáticos abstractos en espacios de dimensión infinita hasta su implementación computacional práctica, pasando por el estudio de arquitecturas específicas y su aplicación a ecuaciones de la física matemática. Se ha hecho una revisión de la bibliografía más destacada sobre el tema y se ha estructurado sistemáticamente en el trabajo.

Los operadores neuronales constituyen un marco orientado al aprendizaje de transformaciones entre espacios de funciones, lo que permite aproximar operadores que actúan sobre soluciones de ecuaciones diferenciales superando las limitaciones de dimensión finita de los modelos tradicionales. Estos operadores tienen un gran futuro por delante puesto que acelerar la resolución y simulación de sistemas físicos complejos puede resultar muy útil en la ciencia y la ingeniería. Primero, presentamos el marco conceptual y el problema de aproximación de operadores entre espacios de funciones, analizando los teoremas de aproximación universal, la convergencia y su comparativa con métodos clásicos (Capítulo 1). Después, los conceptos teóricos se concretaron en una arquitectura, el Operador Neuronal de Fourier (FNO), detallando su estructura algorítmica en el dominio de las frecuencias mediante transformadas discretas (Capítulo 2). Posteriormente, reformulamos diversos problemas clásicos de la física matemática bajo este nuevo paradigma, planteando la resolución de ecuaciones diferenciales como la aproximación directa de su operador solución (Capítulo 3). Por último, describimos la implementación computacional en Python de las metodologías estudiadas, donde se entrena un modelo FNO y validamos empíricamente su rendimiento y capacidad de inferencia mediante diversas métricas (Capítulo 4). Además, se ofrece una visión del potencial práctico de estas herramientas frente a los solucionadores tradicionales.

Uno de los principales retos afrontados durante la elaboración de este trabajo ha sido la escasez de bibliografía disponible, así como el hecho de que mucha de ella estuviera dirigida a un público con un perfil más técnico que teórico. A ello se suma la juventud de este campo de estudio, dado que los primeros trabajos sobre operadores neuronales datan del año 2021. En muchas ocasiones la labor del trabajo ha sido la de formalizar algunos conceptos y componentes arquitectónicas que no resultaban del todo claros en la literatura de aprendizaje profundo. A pesar de todo esto, he conseguido ampliar mi conocimiento sobre el tema que era el principal objetivo al realizar este trabajo. También he desarrollado capacidad de análisis, síntesis e implementación computacional.

# Bibliografía

- [1] Repositorio de GitHub. URL: <https://github.com/neuraloperator/neuraloperator>.
- [2] Yoshua Bengio, Ian Goodfellow, Aaron Courville et al. *Deep learning*. Vol. 1. MIT press Cambridge, MA, USA, 2017.
- [3] Julius Berner, Miguel Liu-Schiaffini, Jean Kossaifi, Valentin Duruisseaux, Boris Bonev, Kamyar Azizzadenesheli y Anima Anandkumar. «Principled approaches for extending neural architectures to function spaces for operator learning». En: *arXiv preprint arXiv:2506.10973* (2025).
- [4] Stephen Boyd y Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. URL: <https://web.stanford.edu/~boyd/cvxbook/>.
- [5] Tianping Chen y Hong Chen. «Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems». En: *IEEE Transactions on Neural Networks* 6.4 (1995), págs. 911-917. DOI: [10.1109/72.392253](https://doi.org/10.1109/72.392253).
- [6] James W. Cooley y John W. Tukey. «An Algorithm for the Machine Calculation of Complex Fourier Series». En: *Mathematics of Computation* 19.90 (1965), págs. 297-301. DOI: [10.1090/S0025-5718-1965-0178586-1](https://doi.org/10.1090/S0025-5718-1965-0178586-1).
- [7] Valentin Duruisseaux, Jean Kossaifi y Anima Anandkumar. «Fourier Neural Operators Explained: A Practical Perspective». En: *arXiv preprint arXiv:2512.01421* (2025).
- [8] *Ecuaciones de Navier-Stokes: Dinámica de Fluidos, Modelado y Análisis*. Modern Physics Insights. URL: <https://modern-physics.org/ecuaciones-de-navier-stokes-dinamica-de-fluidos-modelado-y-analisis/>.
- [9] Jaime Gabriel Vegas. «Redes neuronales diferenciales: Fundamentos y Aplicaciones». En: (2024).
- [10] David J Griffiths. «The Dirac delta function». En: *Introduction to Electrodynamics* (1999), págs. 45-51.
- [11] Grupo Histología. *Neuronas: Estructura y morfología*. <https://histologiaub.blogspot.com/p/morfologia.html>.
- [12] Kaiming He, Xiangyu Zhang, Shaoqing Ren y Jian Sun. «Deep residual learning for image recognition». En: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, págs. 770-778.
- [13] Evelyn Herberg. «Lecture Notes: Neural Network Architectures». En: *arXiv preprint arXiv:2304.05133* (2023).
- [14] Jean Kossaifi, Nikola Kovachki, Zongyi Li, David Pitt, Miguel Liu-Schiaffini, Robert Joseph George, Boris Bonev, Kamyar Azizzadenesheli, Julius Berner, Valentin Duruisseaux et al. «A library for learning neural operators». En: *arXiv preprint arXiv:2412.10354* (2024).

- [15] Jean Kossaifi, David Pitt, Nikola Kovachki, Zongyi Li y Anima Anandkumar. *Neural Operators in PyTorch*. Documentación oficial de neuraloperator 2.0.0. URL: <https://neuraloperator.github.io/dev/index.html>.
- [16] Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart y Anima Anandkumar. «Neural operator: Learning maps between function spaces with applications to pdes». En: *Journal of Machine Learning Research* 24.89 (2023), págs. 1-97.
- [17] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart y Anima Anandkumar. «Fourier neural operator for parametric partial differential equations». En: *arXiv preprint arXiv:2010.08895* (2020).
- [18] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart y Anima Anandkumar. «Neural operator: Graph kernel network for partial differential equations». En: *arXiv preprint arXiv:2003.03485* (2020).
- [19] Peter A. Loeb y Erik Talvila. *Lusin's Theorem and Bochner Integration*. 2004. arXiv: [math/0406370](https://arxiv.org/abs/math/0406370) [math.CA]. URL: <https://arxiv.org/abs/math/0406370>.
- [20] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang y George Em Karniadakis. «Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators». En: *Nature Machine Intelligence* 3.3 (mar. de 2021), págs. 218-229. ISSN: 2522-5839. DOI: [10.1038/s42256-021-00302-5](https://doi.org/10.1038/s42256-021-00302-5). URL: <http://dx.doi.org/10.1038/s42256-021-00302-5>.
- [21] Unique Subedi y Ambuj Tewari. *Is Zero-Shot Super-Resolution Possible in Operator Learning?* 2026. arXiv: [2606.00296](https://arxiv.org/abs/2606.00296) [stat.ML]. URL: <https://arxiv.org/abs/2606.00296>.
- [22] Vladimir N. Vapnik. *Statistical Learning Theory*. New York: Wiley-Interscience, 1998. ISBN: 0-471-03003-1.
- [23] Zero Math AI. *Convolución 1x1: Una operación convolucional que transforma canales sin cambiar el tamaño espacial*. Zero Math AI. 12 de dic. de 2025. URL: <https://zeromathai.com/es/onebyone-conv-es/>.

# Anexo

| Notación                                                                                 | Significado                                                                                                                                                                                                                                                   |
|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Espacios de Sobolev</b>                                                               |                                                                                                                                                                                                                                                               |
| $W^{k,p}(\Omega)$                                                                        | $W^{k,p}(\Omega) = \{u \in L^p(\Omega) : D^\alpha u \in L^p(\Omega) \text{ para todo }  \alpha  \leq k\}.$                                                                                                                                                    |
| $H^k(\Omega)$                                                                            | $H^k(\Omega) = W^{k,2}(\Omega).$                                                                                                                                                                                                                              |
| $H_0^1(\Omega)$                                                                          | $H_0^1(\Omega) = W_0^{1,2}(\Omega) = \{u \in H^1(\Omega) : u = 0 \text{ en } \partial\Omega\}.$                                                                                                                                                               |
| $H^{-1}(\Omega)$                                                                         | $H^{-1}(\Omega) = (H_0^1(\Omega))^*.$                                                                                                                                                                                                                         |
| <b>Operadores neuronales</b>                                                             |                                                                                                                                                                                                                                                               |
| Red Neuronal<br>$N_n(\sigma; \mathbb{R}^d)$                                              | $\left\{f : \mathbb{R}^d \rightarrow \mathbb{R} : f(x) = W_n \sigma(\cdots \sigma(W_0 x + b_0) \cdots) + b_n\right\}.$                                                                                                                                        |
| Operador Integral<br>$\text{IO}(\sigma; D, \mathbb{R}^{d_1}, \mathbb{R}^{d_2})$          | $\left\{f \mapsto \int_D \kappa(\cdot, y) f(y) dy + b : \kappa \in N_{n_1}(\sigma; \mathbb{R}^d \times \mathbb{R}^d, \mathbb{R}^{d_2 \times d_1})\right.$<br>$\left., b \in N_{n_2}(\sigma; \mathbb{R}^d, \mathbb{R}^{d_2}), n_1, n_2 \in \mathbb{N}\right\}$ |
| Operador Neuronal<br>$\mathcal{NO}_n(\sigma; D, D', \mathbb{R}^{d_a}, \mathbb{R}^{d_u})$ | $\left\{f \mapsto S_n \sigma \circ \cdots \circ \sigma S_1(f) : S_i \in \text{IO}(\sigma; D, \mathbb{R}^{d_{i-1}}, \mathbb{R}^{d_i})\right\}$                                                                                                                 |
| <b>Propiedades de funciones de activación</b>                                            |                                                                                                                                                                                                                                                               |

| Notación | Significado                                                                                                                                                                                                     |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $A_m$    | $\{\sigma \in C(\mathbb{R}) : \exists n \in \mathbb{N} \text{ tal que } N_n(\sigma; \mathbb{R}^d)$ <p>es denso en <math>C^m(K)</math>, <math>\forall K \subset \mathbb{R}^d</math> compacto<math>\}</math>.</p> |
| $A_m^L$  | $\left\{ \sigma \in A_m : \sigma \text{ es Borel medible, } \sup_{x \in \mathbb{R}} \frac{ \sigma(x) }{1+ x } < \infty \right\}.$                                                                               |

**Definición 6.** Denotamos por **BA** al conjunto de aplicaciones  $\sigma \in A_0$  tales que, para cualquier conjunto compacto  $K \subset \mathbb{R}^d$ ,  $\varepsilon > 0$ , y  $C \geq \text{diam}(K)$ , existe un número  $n \in \mathbb{N}$  y una red neuronal  $f \in N_n(\sigma; \mathbb{R}^d, \mathbb{R}^d)$  tal que

$$|f(x) - x|_2 \leq \varepsilon, \quad \forall x \in K,$$

$$|f(x)|_2 \leq C, \quad \forall x \in \mathbb{R}^d.$$

**Definición 7** (Operador de rango finito). Sea  $\mathcal{H}$  un espacio de Hilbert y sea  $T : \mathcal{H} \rightarrow \mathcal{H}$  un operador lineal. Se dice que  $T$  es un operador de rango finito si su imagen tiene dimensión finita, es decir,

$$\dim(T(\mathcal{H})) < \infty.$$

En particular, si  $\dim(T(\mathcal{H})) = n$ , entonces existen  $v_1, \dots, v_n \in \mathcal{H}$  y una base ortonormal  $e_1, \dots, e_n$  de  $T(\mathcal{H})$  tales que

$$Tx = \sum_{i=1}^n \langle x, v_i \rangle e_i, \quad x \in \mathcal{H}.$$